Evolutionary Task Discovery: Advancing Reasoning Frontiers via Skill Composition and Complexity Scaling

Liqin Ye[1†] **Yanbin Yin**[1] **Michael Galarnyk**[1] **Yuzhao Heng**[1]
Sudheer Chava[1] **Chao Zhang**[1]
[1] Georgia Institute of Technology

Abstract

The reasoning frontier of Large Language Models (LLMs) has advanced significantly through modern post-training paradigms (e.g., Reinforcement Learning from Verifiable Rewards (RLVR)). However, the efficacy of these methods remains fundamentally constrained by the diversity and complexity of the training data. One practical solution is data synthesis; yet, prevalent methods relying on unstructured mutation or exploration suffer from *homogeneity collapse*, failing to systematically expand the reasoning frontier. To overcome this, we propose **Evo**lutionary **T**ask **D**iscovery (EvoTD), a framework that treats data synthesis as a directed search over a dual-axis manifold of *Algorithmic Skills* and *Complexity Attributes*. We introduce structured evolutionary operators to navigate this space: a `Crossover` operator that synthesizes novel skill compositions to enhance diversity, and a Parametric `Mutation` operator that scales structural constraints (e.g., input size, tree depth) to drive robust generalization. Crucially, we integrate a dynamic *Zone of Proximal Development* filter, ensuring tasks lie within the learnable region of the model. Empirically, EvoTD delivers substantial reasoning gains that generalize consistently across model architectures, pretraining regimes, and scales, demonstrating that structured evolutionary curricula can effectively support reasoning improvement. We release our code on https://github.com/liqinye/EvoTD.

1 Introduction

The reasoning abilities of LLMs have advanced significantly, with models like DeepSeek-R1 [Guo et al., 2025] and Gemini 3 Pro [Team, 2025a] achieving impressive performance on challenging benchmarks such as Humanity's Last Exam [Phan et al., 2025] and AIME [AIME, 2025]. A key driver of this progress is Reinforcement Learning with Verifiable Rewards (RLVR), which leverages programmatically validated signals such as code correctness or solution accuracy, instead of relying on human-annotated examples [Lambert et al., 2024, Guo et al., 2025]. Yet the effectiveness of RLVR is tied to the diversity and quality of the training corpus. Reasoning training relies heavily on costly human-curated question–answer pairs [Cobbe et al., 2021] or noisy web-scraped corpora [Moshkov et al., 2025], both of which remain static and constrain scalability as well as the exploration of reasoning difficulty [Zhang et al., 2025b].

One promising response to these data limitations is data synthesis, where strong LLMs manufacture training samples [Jung et al., 2025]. However, current methods suffer from critical structural limitations. Standard training synthesis approaches [Zhao et al., 2025, Xu et al., 2024, Luo et al., 2024, 2025a] typically rely on unstructured mutation elicited via prompting (e.g., "make this problem

[†]Correspondence to: Liqin Ye ⟨liqiny@gatech.edu⟩, Chao Zhang ⟨chaozhang@gatech.edu⟩

harder", "add more constraints"). This frequently results in generating near-duplicate samples [Shumailov et al., 2023, Kim et al., 2025]. Diversity-aware approaches regularize this by preserving samples with low textual similarity scores (e.g., BLEU distance, ROUGH-L) [Xia et al., 2025, Huang et al., 2025, Wang et al., 2023]; yet, such textual similarity is not necessarily equivalent to the algorithm diversity. A single algorithm can be expressed using completely different linguistic formulations. Recent works have begun to organize synthesis around explicit skill taxonomies [Shah et al., 2024, Zeng et al., 2025, Khan et al., 2025], ensuring that the training distribution covers a diverse set of logical concepts. While these methods successfully categorize *what* logic is required (the skill), they fundamentally neglect *how* that logic is constrained (the complexity). This leads to a "flat" curriculum that covers the breadth of algorithmic concepts but fails to systematically scale the computational complexity to induce generalization. Consequently, there remains a critical gap for a generative framework capable of automatically manipulating both logical backbones and structural scales of reasoning tasks to drive robust, post-training improvement.

In light of this, we propose **Evo**lutionary **T**ask **D**iscovery (EvoTD), a novel data synthesis framework that treats task generation as a directed search over a structured abstraction space. As the total complexity of a task is a product of both intrinsic complexity of the general logic and the extrinsic constraints of the problem instance [Smith-Miles and Lopes, 2012, Doctor et al., 2023], we conceptualize a reasoning task as a composition of two orthogonal dimensions: *Algorithmic Skills* (the logic backbone, e.g., binary search, heaps) and *Complexity Attributes* (the structural constraints, e.g., input size, graph size). Rather than relying on black-box prompting to "make this diverse and harder", we introduce a principled evolutionary task discovery mechanism. Motivated by Evolutionary Algorithms [Holland, 1973], we formalize a `Attribute Mutation` operator (which scales complexity attributes) and a `Skill Crossover` operator (which explores novel skill combinations) to systematically traverse the task space. Our framework adopt a proposer-solver paradigm where the proposer synthesizes tasks and the solver learns from the tasks. To ensure the task falls within solver's *Zone of Proximal Development* (ZPD) [Vygotsky, 1978], we retain tasks that are neither trivial nor impossible. Crucially, because learnability is assessed against the current solver policy π_θ, our filter implements a dynamic curriculum: as the solver improves, previously impossible tasks become learnable, and the training distribution automatically shifts toward harder instances. Complementing this learnability filter, we leverage LLM's metacognitive capability [Didolkar et al., 2024] to verify the skill alignment of synthesized tasks. Through this rigorous cycle of evolutionary proposal and filtering, EvoTD constructs a diverse, adaptive curriculum that continuously pushes models' reasoning frontier.

Empirical evaluations across five diverse benchmarks, spanning code generation and mathematical reasoning, demonstrate that EvoTD significantly outperforms standard heuristic synthesis paradigms. Notably, on the Qwen3-4B reasoning model, our method achieves substantial gains on the AIME 24 (+8.2%) and AIME 25 (+7.2%) benchmarks. We further show that our method generalizes across three orthogonal axes of model variation: architecture (Qwen3, LLaMA), pretraining regime (Base, Instruct), and parameter scale (3–8B), attaining the best performance on every evaluated backbone. Beyond code and math, gains transfer cleanly to general-domain reasoning benchmarks (MMLU-Pro, SuperGPQA), with the largest improvements concentrated on STEM-adjacent disciplines where rigorous reasoning is the binding constraint.

We make the following contributions:

- We formalize reasoning task synthesis as directed traversal over a dual-axis manifold: *Algorithmic Skills* and *Complexity Attributes*. This abstraction moves beyond naive prompting, allowing control of logic and constraints to generate quality reasoning curricula.
- We introduce two evolutionary operators for task synthesis: `Attribute Mutation` for complexity scaling and `Skill Crossover` for composition discovery. This dual-operator design empowers EvoTD to systematically explore the boundaries of the task space.
- We conduct extensive evaluations across five benchmarks including code generation and mathematical reasoning, demonstrating the utility of our framework.

2 EvoTD

In this section, we introduce EvoTD, an automated framework designed to systematically expand the diversity and complexity of reasoning tasks through the learning process. We begin by formalizing

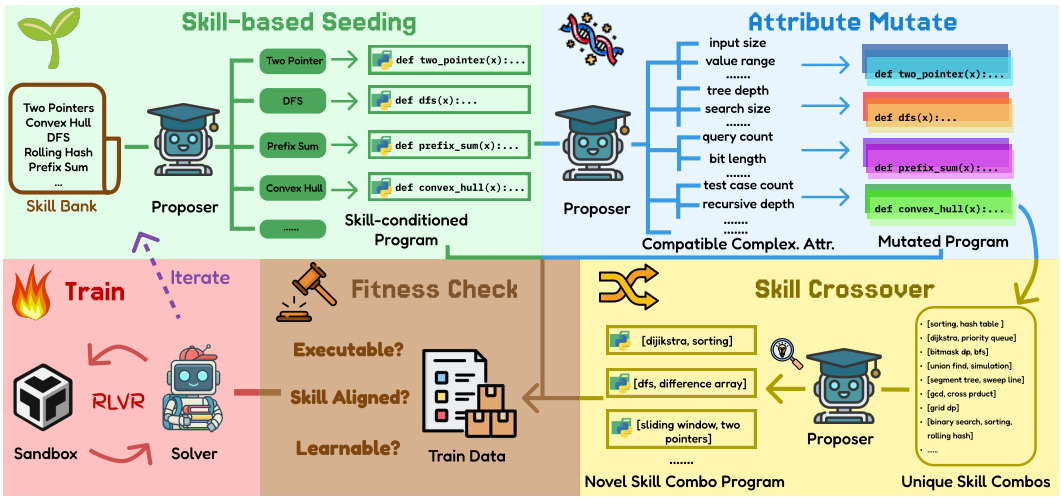


Figure 1: EvoTD Framework. (1) Skill-based Seeding: generate skill conditioned programs. (2) Complexity Attribute Mutation: mutate existing programs based on applicable complexity attributes. (3) Skill Crossover: synthesize programs with new skill combinations. (4) Fitness Check: verify programs via executability, skill alignment, and learnability. (5) Train: train solvers with valid tasks.

the space of executable reasoning tasks across deduction, abduction, and induction modes (§2.1). We then define our dual-axis abstraction, which explicitly decouples *algorithmic skills* from *structural complexity* to enable precise evolutionary control (§2.2). Finally, we detail the evolutionary synthesis engine, describing how attribute mutation and skill crossover operators cooperatively traverse this space to discover novel training samples (§2.3). Our training pipeline is built upon verifier-based RL paradigm; formal background is provided in Appendix A. The complete procedural flow of our framework is outlined in Figure 1 and Algorithm 1.

2.1 Problem Formulation

To study how training data can systematically *push* an LLM's algorithmic reasoning frontier, we need to first define the reasoning task. We follow the same task formalization as Zhao et al. [2025]. Let $\mathcal{P}$ denote a program space, $\mathcal{I}$ an input space, and $\mathcal{O}$ an output space. An algorithmic reasoning task can be represented as a program-input-output triplet $(p, i, o) \in \mathcal{P} \times \mathcal{I} \times \mathcal{O}$, where $o = p(i)$ is obtained by executing p on i. We define distinct *modes of reasoning* by which elements of the triplet are observed versus predicted:

- **Deduction** $\mathcal{T}_{\text{ded}}$: Given a program p and an input i, predict the output o. This tests the model's forward thinking to simulate logic step-by-step, $(p, i \to o)$.
- **Abduction** $\mathcal{T}_{\text{abd}}$: Given a program p and an output o, infer a valid input i such that $p(i) = o$. This requires reverse-engineering the logical constraints, $(p, o \to i)$.
- **Induction** $\mathcal{T}_{\text{ind}}$: Given a natural language problem statement m and a set of input-output pairs $\{(i_n, o_n)\}_{n=1}^{N}$, synthesize a program p that satisfies $p(i_n) = o_n$ for all n. This tests the model's ability to infer executable rules from specifications and observations, $(m, \{(i_n, o_n)\}_{n=1}^{N} \to p)$.

Executable Task Instantiation Each synthesized task is grounded in executable code, an expressive and verifiable representation for reasoning problems. To ensure all tasks are grounded, we follow the generative procedure from Zhao et al. [2025]. We employ a Python code executor as an oracle. For Deduction and Abduction, the task proposer synthesizes a program-input pair (p, i) and executes it to obtain the deterministic output o. The solver is then presented with the respective partial tuple ((i, p) for Deduction and (p, o) for Abduction) to predict the corresponding missing part. For Induction, we utilize the pool of verified programs generated during Deduction and Abduction phases. The proposer conditions on an existing program p to generate a natural language problem statement m and a set of input-output pairs $\{(i_n, o_n)\}_{n=1}^{N}$. The solver is provided with a problem prompt and a subset of input-output pairs, while the remainder pairs serve as hidden test cases to verify the correctness of the solver-synthesized program.

2.2 Task Abstraction

To enable evolutionary discovery, we first establish a structured representation of the task space. Direct mutation of raw problem specifications is often inefficient, as the space of valid coding tasks is sparse and highly discontinuous [Luo et al., 2025b, Orvalho and Kwiatkowska, 2025]. Random textual perturbations often fail to induce meaningful shifts in algorithmic reasoning, instead producing syntactically malformed code or preserving original computational complexity [Zhang et al., 2025a, Novikov et al., 2025, Abed et al., 2025]. Consequently, standard evolutionary approaches which rely on unstructured mutation struggle to discover novel regions of the solution space without explicit semantic guidance [van Stein et al., 2025, Hemberg et al., 2024]. Therefore, we propose to control task evolution through a low-dimensional abstraction that decouples the *semantic logic* of a coding task from its *computational structure*.

We conceptualize a coding task t as a point in a dual-axis manifold $\mathcal{T} \approx \mathcal{S} \times \mathcal{C}$, where $\mathcal{S}$ represents **Algorithmic Skills** and $\mathcal{C}$ represents **Complexity Attributes**. We describe these two axes as follows:

The Semantic Axis: Algorithmic Skills ($\mathcal{S}$) Reasoning is inherently compositional. Complex problems are rarely monolithic: they are aggregates of atomic logical primitives [Piantadosi et al., 2016, Lippl et al., 2025]. We define a "Skill" $s \in \mathcal{S}$ as a fundamental algorithmic pattern required to solve a problem (e.g. *two pointers, dijkstra*). By abstracting tasks into skills, we transform the opaque goal of "diversity" into a measurable coverage problem. This allows our evolutionary operators to target specific "blind spots" in model's skill set, rather than hoping for diversity through randomness.

The Structural Axis: Complexity Attributes ($\mathcal{C}$) While the algorithmic skills establish the *qualitative* reasoning method, the complexity attributes dictate the *quantitative* magnitude of the problem instance. A *Depth-First Search* task can be trivial on a tree of depth 3 but challenging on a graph with cycles and 10^4 nodes. We define "Complexity Attributes" $c \in \mathcal{C}$ as the set of parametric constraints governing the execution environment (e.g. *input size, value range*). LLMs often learn brittle heuristics for simple coding instances that fail to generalize as problem scale increases [Power et al., 2022]. By treating $\mathcal{C}$ as an independent axis, we can force the model to move beyond superficial pattern matching toward scale-invariant algorithmic solutions.

The candidate lists for $\mathcal{S}$ and $\mathcal{C}$ can be populated via expert curation, LLM synthesis, or automated extraction. While our framework is agnostic to the source of these axes, in this work, we adopt a data-driven initialization, leveraging LLM metacognitive ability [Didolkar et al., 2024] to extract from a seed corpus [Shi et al., 2024]. This approach offers an optimal balance between scalability and domain-specific fidelity. It circumvents the bottleneck of human annotation while anchoring the quality of skills directly in rigorous, real-world problem distributions. The extracted skill bank and complexity attributes are detailed in Figure 8 and 9.

2.3 Evolutionary Task Synthesis

Given this orthogonal dual-axis abstraction, we propose an evolutionary synthesis engine to systematically traverse this manifold. Let $\Phi(t) = (s, \mathbf{c})$ represent the projection of a task into its skill and complexity attributes. We design two distinct operators: `Attribute Mutation` and `Skill Crossover`, aiming for population diversity and frontier expansion. We manipulate these two task axes to generate novel training samples, filtered by a rigorous multi-objective verification protocol.

Skill-based Seeding We initialize the population $\mathcal{D}_{\text{skill}}$ by instantiating a baseline task for each unique skill $s \in \mathcal{S}$ in our skill bank, which we extract from a seed dataset [Shi et al., 2024]. During the synthesis of Deduction and Abduction tasks, the generation of program-input pair (p, i) is explicitly conditioned on the target skill. The proposer is prompted to generate a program p that implements the core algorithmic logic of s (See Figure 14 and 18). This guarantees that our evolutionary search begins with maximal semantic coverage of the foundational skill space.

`Attribute Mutation` The mutation operator drives the "vertical" evolution of complexity by intensifying structural constraints while strictly preserving the logical backbone s. Unlike static scaling, $\mathcal{M}_{\text{attr}}$ leverages the proposer LLM's metacognitive ability to perform an attribute compatibility audit. Formally, given a parent program $p \in \{\mathcal{T}_{\text{ded}}, \mathcal{T}_{\text{abd}}\}$ synthesized from skill-based seeding stage, the proposer first selects what attributes are applicable: $\pi_{\text{prop}}(p) \to \mathbf{c}^*$, where $\mathbf{c}^* \in 2^{\mathcal{C}}$ is a subset of

attributes that are valid to be scaled in p (e.g., selecting *tree depth* only for p containing tree). The mutation is then defined as:

$$\mathcal{M}_{\text{attr}}(t, \pi_{\text{prop}}) = t' \quad \text{s.t.} \quad \Phi(t') = (s, \mathbf{c} + \Delta\mathbf{c}') \tag{1}$$

where t is a task instance and $\Delta\mathbf{c}'$ represents the directed intensification of the selected attributes by the. For each parent, we generate a diverse cohort of n variants ($n = 10$ in this work), each embodying a distinct combinatorial intersection of the LLM-selected complexity attributes.

`Skill Crossover` The crossover operator $\mathcal{X}_{\text{skill}} : 2^{\mathcal{S}} \to \mathcal{T}$ enables horizontal exploration through combinatorial logic synthesis. Let $\Lambda(\mathcal{D}) = \{S_1, S_2, \ldots, S_n\}$ be the set of skill combinations currently realized in the population via proposer LLM's metacognitive labeling during filtering process (elaborated in the following paragraph). The operator performs a novelty combinatorial search to generate a task t_{new} possessing undiscovered skill combinations:

$$\mathcal{X}_{\text{skill}}(\Lambda(\mathcal{D}), \pi_{\text{prop}}) = t_{\text{new}} \quad \text{s.t. Skill}(t_{\text{new}}) = S_{\text{new}} \notin \Lambda(\mathcal{D}) \tag{2}$$

Because arbitrary permutations of skills can yield conceptually incongruent tasks, we prevent combinatorial degeneration by explicitly prompting the proposer to discover naturally synergistic skill combinations. To ensure this integration is not merely superficial, we further enforce that all selected skills must be essential and deeply interconnected within generated tasks, rather than appearing sequentially (see Figures 20 and 16). By targeting the sparse regions of the meaningful skill-composition set, $\mathcal{X}_{\text{skill}}$ prevents logic stagnation and forces the model to synthesize increasingly novel, multi-skill reasoning tasks.

Multi-objective Fitness Check To ensure the synthesized task is both computationally valid and pedagogically instructive, we enforce a rigorous verification protocol $\mathcal{V}(t)$ modeled as a composite indicator function of three hierarchical objectives as below:

$$\mathcal{V}(t) = v_{\text{exec}}(t) \cdot v_{\text{skill}}(t) \cdot v_{\text{learn}}(t) \tag{3}$$

1. **Executability (v_{exec}):** For $\mathcal{T}_{\text{ded}}$ and $\mathcal{T}_{\text{abd}}$, we confirm that the program-input pair (p, i) is free of syntax errors and terminates within limits by a Python executor. For $\mathcal{T}_{\text{ind}}$, we validate that input-output pairs $(i_n, o_n)\}_{n=1}^{N}$ are faithful by the ground-truth program p.
2. **Skill Alignment (v_{skill}):** we perform a task's skill audit to ensure the generated task strictly adheres to the requested algorithmic requirements. The task is retained iff the target skill is preserved in the detected skill sets $\hat{\mathcal{S}}$: $v_{\text{align}}(t) = \mathbb{I}[s_{\text{target}} \in \hat{\mathcal{S}}]$.
3. **Learnability (v_{learn}):** We assess the empirical difficulty using the current solver policy π_θ's pass rate over k attempts. We strictly filter for non-trivial yet solvable instances (avg@$k \in (0, 1)$):

$$v_{\text{learn}}(t) = \mathbb{I}[0 < \mathbb{E}_{k \sim \pi_\theta}[\text{solved}(t_k)] < 1] \tag{4}$$

3 Experiments

3.1 Experimental Setup

Implementation Details We evaluate EvoTD on two model types as our solver: 1) Base models: Qwen3-4B-Base and Qwen3-8B-Base [Team, 2025c] and 2) Reasoning models: Qwen3-4B with thinking enabled [Team, 2025c]. Our training pipeline is built based on VeRL framework [Sheng et al., 2025]. We adopt o4-mini [OpenAI, 2026] as the proposer in our main experiments. To assess EvoTD's sensitivity to this choice, Appendix C.2 reports an ablation with two open-weight proposers, gpt-oss-120b and deepseek-r1-0528. To efficiently leverage the synthesized data and modulate the difficulty, the proposer is prompted to inject natural language hints to the induction task which is too opaque (*avg*@k= 0). This helps bootstrap more samples falling within solver's ZPD. For additional implementation details, see Appendix G.

Datasets To evaluate the model's ability to generalize beyond its training distribution, we conduct evaluations across five benchmarks spanning code generation and mathematical reasoning: 1) Code Generation: MBPP+ [Austin et al., 2021] for rigorous correctness checking; LiveCodeBench v6 [Jain et al., 2025] to assess contamination-free problems. 2) Math Reasoning: AIME 24 and 25 [AIME, 2025] to probe the frontier of competitive problem-solving; OlympiadBench [He et al., 2024] to assess logical deduction across diverse domains. 3) General Domain Reasoning: MMLU-Pro [Wang et al., 2024] for robust multi-disciplinary reasoning; SuperGPQA [Team, 2025b] for graduate-level expertise across disciplines. For additional details on datasets, see Appendix B.

Table 1: Results on Thinking models across code and math benchmarks. **Bold** means peak performance. Underline means second-best.

Model	LCB^{v6}	MBPP^{+}	AME24	AME25	Olympiad	CAvg	MAvg	AVG
Pass@1								
Qwen3-4B (Thinking)	42.9	51.3	34.0	21.8	45.6	47.1	33.8	39.1
+Evol-Instruct	47.4	53.7	35.4	22.4	46.9	50.6	34.9	41.2
+EvoTD (Iter1)	46.8	54.7	36.7	25.8	47.3	50.8	36.6	42.3
+EvoTD (Iter2)	48.5	**56.2**	37.9	28.3	49.1	52.4	38.4	44.0
+EvoTD (Iter3)	**49.8**	55.8	**42.2**	**29.0**	**49.4**	**52.8**	**40.2**	**45.2**
Pass@8								
Qwen3-4B (Thinking)	53.8	68.3	55.4	39.0	59	56.4	51.1	55.1
+Evol-Instruct	58.0	54.8	55.0	44.2	59.1	56.4	52.8	54.2
+EvoTD (Iter1)	58.9	70.9	60.2	47.3	59.7	64.9	55.7	59.4
+EvoTD (Iter2)	60.8	**72.8**	61.4	47.6	61.5	**66.8**	56.8	60.8
+EvoTD (Iter3)	**61.5**	71.2	**63.2**	**49.9**	**62.1**	66.4	**58.4**	**61.6**

Table 2: Results on Base and Instruct models across code and math benchmarks.

Model	LCB^{v6}	MBPP^{+}	AME24	AME25	Olympiad	CAvg	MAvg	AVG
Qwen3-4B-Base	20.6	53.7	9.1	3.3	26.2	37.2	12.9	22.6
+Evol-Instruct	23.2	52.1	10.0	10.0	27.0	37.7	15.7	24.5
+SPIRAL[1]	27.3	61.4	10.0	**13.3**	**40.0**	44.4	21.1	30.4
+Agent0	**27.6**	**63.5**	10.0	3.3	34.7	**45.6**	16.0	27.8
+EvoTD (Ours)	26.4	58.5	**19.5**	10.0	39.6	42.5	**23.0**	**30.8**
Qwen3-8B-Base	29.6	58.3	13.3	6.7	34.8	44.0	18.3	28.5
+Evol-Instruct	31.9	59.8	11.4	9.6	40.1	45.9	20.4	30.6
+SPIRAL[1]	28.8	58.7	14.2	**19.6**	**42.5**	43.8	**25.4**	32.8
+Agent0	32.0	**71.2**	13.1	10.1	41.0	**51.6**	21.4	33.5
+EvoTD (Ours)	**32.8**	65.3	**16.7**	16.7	**42.5**	49.1	25.3	**34.8**
LLaMA-3.2-3B-Instruct	11.0	25.9	3.3	0.0	11.0	18.5	4.8	10.2
+Evol-Instruct	11.1	25.4	**10.0**	0.0	12.3	18.3	7.4	11.8
+Agent0	**11.6**	31.5	3.3	0.0	11.0	21.6	4.8	11.5
+EvoTD (Ours)	10.7	**42.1**	**10.0**	0.0	**14.8**	**26.4**	**8.3**	**15.5**
LLaMA-3.1-8B-Instruct	16.6	59.8	3.3	0.0	15.4	38.2	6.2	19.0
+Evol-Instruct	15.7	61.1	3.3	3.3	**18.4**	38.4	8.3	20.4
+SPIRAL[1]	16.7	60.6	0.8	3.3	17.5	38.7	7.2	19.8
+Agent0	16.1	61.1	**10.0**	0.0	16.0	38.6	8.7	20.6
+EvoTD (Ours)	**17.6**	**62.4**	6.7	**6.7**	17.6	**40.0**	**10.3**	**22.2**

Metrics For Base and Instruct models, we employ greedy decoding to report Pass@1, with the exception of the AIME benchmarks, where we duplicate each problem 32 times to report *mean@32*. For Thinking models, we align with the evaluation protocol of Team [2025c], using temperature $T = 0.6$ and top-$p = 0.95$. We generate $k = 8$ responses per question across all benchmarks (increased to $k = 32$ for AIME) and report both Pass@1 and Pass@8.

Baselines We benchmark EvoTD against a spectrum of data synthesis paradigms, ranging from heuristic mutation to self-improving. For experiments on base models, we compare against three distinct categories: (1) The Base Model without any finetuning; (2) Evol-Instruct [Luo et al., 2024, Zhao et al., 2025], which represents the standard paradigm of using heuristic prompts to generate data; and (3) Self-Evolving Frameworks: SPIRAL [Liu et al., 2025] which refines reasoning via self-play zero-sum games, and Agent0 [Xia et al., 2025] which self-enhances with tool integration. For reasoning model experiments, we limit our comparison to the Base thinking model and Evol-Instruct.

[1] We use the checkpoint Spiral-Qwen3-4B-Multi-Env and Spiral-Qwen3-8B-Multi-Env available on HuggingFace.

3.2 Main Results

We evaluate EvoTD across two regimes: reasoning-tuned "Thinking" backbones (Table 1) and Base/Instruct backbones (Table 2). Complementary results on general-domain benchmarks (MMLU-Pro, SuperGPQA) are reported in Appendix C.1. The first regime tests whether evolutionary task synthesis can extend the frontier of models that already possess strong reasoning capability. The second probes generalization across model families, training regimes, and scales, benchmarking against state-of-the-art self-evolving methods.

Efficacy on Reasoning Models. We highlight three distinct dimensions of improvement that validate our evolutionary hypothesis: **(1) Holistic Cross-Domain Generalization.** Unlike standard synthesis methods that often incur an "alignment tax" [Lin et al., 2024] where optimizing for math degrades coding capability, EvoTD produces simultaneous improvements on in-domain coding (+5.7% CAvg) and out-of-domain mathematics (+6.4% MAvg), This confirms that our *Skill-based Seeding* and `Skill Crossover` surface abstract logical primitives that transfer across syntactic and symbolic reasoning modalities. **(2) Resilience to Homogeneity Collapse.** A canonical failure mode of distillation-style training is mode collapse, where the model overfits to a single solution path, inflating Pass@1 while degrading Pass@k ($k > 1$). Evol-Instruct exhibits exactly this pathology: its Pass@8 average (54.2%) falls below the untuned backbone (55.1%) despite a higher Pass@1. EvoTD instead achieves consistent performance gains in both Pass@1 (+4%) and Pass@8 (+7.4%), evidence that our diversity-driven curriculum proliferates reasoning trajectories rather than narrowing them. **(3) Amplified Gains on Harder Benchmarks.** The margin between EvoTD and the baseline widens with problem difficulty, reaching +8.2% on AIME24 and +7.2% on AIME25. This trend substantiates the role of our *Complexity Attribute* axis: training on parametrically more complex mutations teaches the model to disentangle core algorithmic reasoning from superficial instance scale, improving robustness on reasoning-intensive tasks.

Generalization across Architectures, Regimes, and Scales. We further demonstrate that EvoTD transfers robustly across three orthogonal axes of model variation: architecture (Qwen3 vs. LLaMA), pretraining regime (Base vs. Instruct), and parameter scale (3/4B vs. 8B). EvoTD attains the highest overall Pass@1 on every one of the four resulting backbones, indicating that the evolutionary curriculum is not bound to a particular model family, training stage, or size. Notably, this holds even against two resource-intensive self-evolving baselines: SPIRAL which requires curated zero-sum self-play environments, and Agent0 which utilizes external tools. EvoTD leads the next-best method on each backbone by +0.4%, +1.3%, +3.7%, and +1.6%, respectively. Together, these confirm that EvoTD offers a pragmatic way for reasoning enhancement, proving that complexity modulation allows base models to scale efficiently without the prohibitive computational overhead of self-training.

Table 3: Evolutionary Operators Ablation

Method	CAvg	MAvg	AVG
Qwen3-4B Thinking *Pass@1*			
EvoTD	52.8	**40.2**	**45.2**
w/o Attribute Mutation	52.7	38.1	43.9
w/o Skill Crossover	**53.1**	38.6	44.4
Qwen3-4B Thinking *Pass@8*			
EvoTD	66.4	**58.4**	**61.6**
w/o Attribute Mutation	66.3	56.6	60.5
w/o Skill Crossover	**67.2**	57.1	61.1

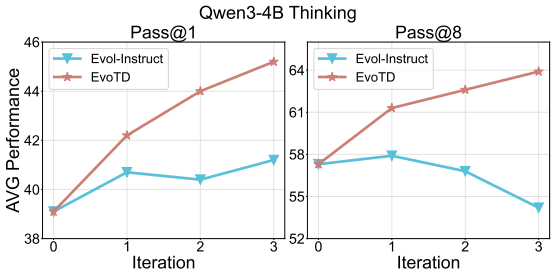
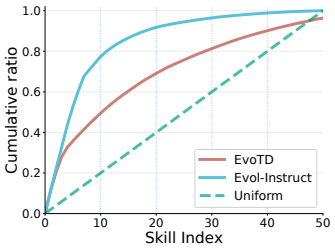


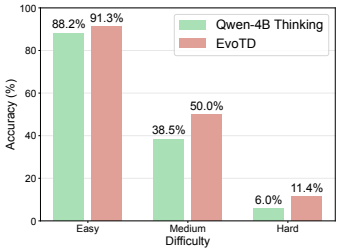
Figure 2: Scaling performance comparison between Evol-Instruct and EvoTD in Pass@1 and Pass@8.

3.3 Analysis

To decompose the mechanisms driving EvoTD's strong performance, we conduct a fine-grained main analysis along four critical dimensions: the necessity of each evolutionary operator via ablation, the iteration dynamics of performance scaling, the skill coverage and compositional novelty, and the difficulty modulation induced by attribute mutation. We also provide qualitative examples for `Attribute Mutation` and `Skill Crossover` in Appendix H.



(a) Skill frequency CDF



(b) LCB^{v6} performance breakdown

Iter	EvoTD	Evol-Instruct
1	**41.8%**	33.2%
2	**43.5%**	36.4%
3	**43.1%**	33.5%
Overall	**32.7%**	24.2%

(c) Unique skill combination ratio

Figure 3: (a) shows the cumulative distribution of skill frequencies, where the uniform reference indicates perfectly balanced skill coverage. (b) shows the detailed performance breakdown on LiveCodeBench^{v6}. (c) shows unique skill combination ratio over iterations.

Ablation of Evolutionary Operators To disentangle the individual contributions of our dual-axis design, we ablate the `Attribute Mutation` and `Skill Crossover` operators. As shown in Table 3, removing `Attribute Mutation` causes a sharp degradation in mathematical reasoning (ΔPass@1 of -2.1%), confirming that complexity scaling is prerequisite for deep, scale-invariant logic. Similarly, eliminating `Skill Crossover` leads to a notable decline in aggregate performance, indicating that combinatorial expansion is essential for broad generalization. We include a more detailed experiment results and discussion in Appendix D.5.

Better Scaling of Iterative Evolution As illustrated in Figure 2, EvoTD demonstrates robust, monotonic scaling in both Pass@1 and Pass@8 metrics throughout the iterative process. In contrast, the Evol-Instruct baseline exhibits early saturation, with performance plateauing after the first iteration. Notably, in the multi-sample regime (Pass@8), the baseline suffers performance degradation in later stages (dropping from 58% to 54%), a phenomenon indicator of mode collapse. Conversely, EvoTD maintains a positive gradient, confirming that our structured, verified evolution is essential for sustaining long-term learning gains.

Skill Uniformity and Composition Coverage To validate the efficacy of *Skill-based Seeding* and `Skill Crossover`, we analyze the topological diversity of the synthesized task manifold. We first quantify semantic balance by examining the Cumulative Distribution Function (CDF) of skill frequencies (See Figure 3a). EvoTD aligns significantly closer to a uniform ideal than baselines, confirming that our seeding strategy maximizes the semantic entropy of the training signal. Unlike Evol-Instruct, it prevents mode collapse into dominant patterns. Complementing this, we measure unique skill combination ratio, a proxy for combinatorial novelty. Table 3c reveals that the crossover operator consistently drives a novelty search, synthesizing a significantly higher volume of previously unseen skill combinations. Together, these metrics confirm that EvoTD does not merely scale data volume, but actively expands the combinatorial frontier of the curriculum, ensuring the model is exposed to structurally diverse reasoning primitives.

Complexity Modulation via Attribute Mutation Finally, a core hypothesis of EvoTD is that evolving complexity attributes ($c \in \mathcal{C}$) drives the model to master harder instances of the same logic. To verify this, we measure the empirical difficulty of mutated tasks compared to their seeds. The mutation operator consistently lowers the solver's success rate, reducing *mean@10* from 56.7% to 51.4%. Crucially, this exposure to "harder" training distributions translates directly to superior generalization on rigorous benchmarks. As illustrated in Figure 3b, EvoTD achieves its largest relative gains (of 90%) on the "Hard" subset of LiveCodeBench^{v6}.

Additional Analyses Beyond the four dimensions above, we provide complementary studies in Appendix D: our extracted skills' coverage and quality (D.2), EvoTD's sensitivity to the initial skill bank(D.1), the fidelity of synthesized crossover tasks(D.3), and the alignment between target skills and the actual solutions(D.4). We also provide a cost analysis of our framework, including latency, API cost, and synthesis efficiency in Appendix F.

4 Related Work

4.1 Evolutionary Algorithms in ML

Recent work has revived evolutionary search to harness LLMs under verifiable feedback, transitioning from single-pass generation to iterative refinement [Novikov et al., 2025, Surina et al., 2025]. Frameworks such as FunSearch [Romera-Paredes et al., 2024] and AlphaEvolve [Novikov et al., 2025] demonstrate that LLM-guided evolution can discover novel mathematical constructions and optimize complex algorithmic artifacts. Beyond code, LLMs increasingly serve as evolutionary operators across diverse domains, from combinatorial and physical engineering optimization [Liu et al., 2024, Brahmachary et al., 2025] to scientific discovery spaces like molecular search and biomedical hypothesis refinement [Wang et al., 2025, Gottweis et al., 2025]. Crucially, these prior approaches primarily employ evolution as an inference-time search mechanism to optimize specific candidate solutions or hypotheses. In contrast, our method shifts the evolutionary paradigm toward data synthesis: we evolve the training distribution itself to automatically construct a progressively rigorous reasoning curriculum.

4.2 Data Synthesis

Synthetic data has become indispensable for scaling post-training beyond the limits of human-authored supervision. Evol-Instruct family (e.g., WizardLM [Xu et al., 2024], WizardCoder [Luo et al., 2024], WizardMath [Luo et al., 2025a]) scale diversity and difficulty through iterative, heuristic rewriting. Recent frameworks integrate stronger structural and behavioral feedback: Prismatic Synthesis aligns diversity with model behavior rather than surface form. DataEnvGym [Khan et al., 2025], ACES [Pourcel et al., 2024], EvalTree [Zeng et al., 2025], and REASONING GYM [Stojanovski et al., 2025] introduce weakness-aware or procedurally controlled generation. However, these approaches remain bottlenecked by rigid taxonomies, manually specified generators, or unstructured textual heuristics, lacking a unified mechanism to jointly modulate compositional logic and parametric complexity. By coupling combinatorial skill crossover with targeted complexity mutation, EvoTD autonomously constructs a dynamic, capability-adaptive reasoning curriculum.

4.3 Self-Improving LLM

A parallel line of research advances self-improving LLMs by closing the loop between autonomous task generation and policy optimization. Frameworks such as Absolute Zero [Zhao et al., 2025] and R-Zero [Huang et al., 2025] demonstrate that models can bootstrap reasoning capabilities from zero external data via verifiable rewards and challenger–solver co-evolution. Similarly, SPIRAL [Liu et al., 2025] casts self-improvement as multi-turn, zero-sum self-play, while Agent0 [Xia et al., 2025] further augments curriculum generation with tool-integrated execution. While these paradigms successfully reduce reliance on human supervision, their innovations center primarily on interaction protocols (competition, self-play, or tool augmentation) leaving the underlying representation of the generated task space largely opaque. In contrast, EvoTD imposes a rigorous structural prior on self-improvement. By evolving tasks across a transparent skill-complexity manifold, we construct a curriculum that is both frontier-seeking and interpretable.

5 Conclusion

We presented EvoTD, a framework that defines task synthesis as a directed evolutionary search over a structured skill-complexity manifold. By formalizing `Attribute Mutation` and `Skill Crossover` operators, EvoTD moves beyond heuristic prompting to systematically traverse the task space, effectively decoupling algorithmic logic from instance complexity. Empirical evaluations across code generation and frontier mathematical reasoning benchmarks show that our method establishes a formidable performance for reasoning models while generalizing across different architectures, regimes, and scales. Overall, our findings confirm that structural evolution provides a far more reliable training signal for complex reasoning than the unconstrained heuristic generation common in standard prompting.

References

Amal Abed, Ivan Lukic, Jörg K. H. Franke, and Frank Hutter. Increasing LLM coding capabilities through diverse synthetic coding tasks. *CoRR*, abs/2510.23208, 2025. doi: 10.48550/ARXIV.2510.23208. URL https://doi.org/10.48550/arXiv.2510.23208.

AIME. American invitational mathematics examination (aime) problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2025. Accessed: 2025-05.

Jacob Austin, Augustus Odena, Maxwell I. Nye, and et al. Program synthesis with large language models. *CoRR*, abs/2108.07732, 2021. URL https://arxiv.org/abs/2108.07732.

Shuvayan Brahmachary, Subodh M. Joshi, Aniruddha Panda, and et al. Large language model-based evolutionary optimizer: Reasoning with elitism. *Neurocomputing*, 622:129272, 2025. doi: 10.1016/J.NEUCOM.2024.129272. URL https://doi.org/10.1016/j.neucom.2024.129272.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, and et al. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168, 2021. URL https://arxiv.org/abs/2110.14168.

Aniket Didolkar, Anirudh Goyal, Nan Rosemary Ke, and et al. Metacognitive capabilities of llms: An exploration in mathematical problem solving. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/2318d75a06437eaa257737a5cf3ab83c-Abstract-Conference.html.

Katarina Doctor, Christine Task, Eric J. Kildebeck, and et al. Toward defining a domain complexity measure across domains. *CoRR*, abs/2303.04141, 2023. doi: 10.48550/ARXIV.2303.04141. URL https://doi.org/10.48550/arXiv.2303.04141.

Juraj Gottweis, Wei-Hung Weng, Alexander N. Daryin, and et al. Towards an AI co-scientist. *CoRR*, abs/2502.18864, 2025. doi: 10.48550/ARXIV.2502.18864. URL https://doi.org/10.48550/arXiv.2502.18864.

Daya Guo, Dejian Yang, Haowei Zhang, and et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638, 2025. doi: 10.1038/S41586-025-09422-Z. URL https://doi.org/10.1038/s41586-025-09422-z.

Chaoqun He, Renjie Luo, Yuzhuo Bai, and et al. Olympiadbench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 3828–3850. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.211. URL https://doi.org/10.18653/v1/2024.acl-long.211.

Erik Hemberg, Stephen Moskal, and Una-May O'Reilly. Evolving code with a large language model. *Genet. Program. Evolvable Mach.*, 25(2):21, 2024. doi: 10.1007/S10710-024-09494-2. URL https://doi.org/10.1007/s10710-024-09494-2.

John H. Holland. Genetic algorithms and the optimal allocation of trials. *SIAM J. Comput.*, 2(2):88–105, 1973. doi: 10.1137/0202009. URL https://doi.org/10.1137/0202009.

Chengsong Huang, Wenhao Yu, Xiaoyang Wang, and et al. R-zero: Self-evolving reasoning LLM from zero data. *CoRR*, abs/2508.05004, 2025. doi: 10.48550/ARXIV.2508.05004. URL https://doi.org/10.48550/arXiv.2508.05004.

Naman Jain, King Han, Alex Gu, and et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=chfJJYC3iL.

Jaehun Jung, Seungju Han, Ximing Lu, and et al. Prismatic synthesis: Gradient-based data diversification boosts generalization in LLM reasoning. *CoRR*, abs/2505.20161, 2025. doi: 10.48550/ARXIV.2505.20161. URL https://doi.org/10.48550/arXiv.2505.20161.

Zaid Khan, Elias Stengel-Eskin, Jaemin Cho, and Mohit Bansal. Dataenvgym: Data generation agents in teacher environments with student feedback. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=00SnKBGTsz.

Jungwoo Kim, Minsang Kim, and Sungjin Lee. Sedi-instruct: Enhancing alignment of language models through self-directed instruction generation. *CoRR*, abs/2502.04774, 2025. doi: 10.48550/ARXIV.2502.04774. URL https://doi.org/10.48550/arXiv.2502.04774.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, and et al. Tülu 3: Pushing frontiers in open language model post-training. *CoRR*, abs/2411.15124, 2024. doi: 10.48550/ARXIV.2411.15124. URL https://doi.org/10.48550/arXiv.2411.15124.

Ziheng Li, Zexu Sun, Jinman Zhao, and et al. Staying in the sweet spot: Responsive reasoning evolution via capability-adaptive hint scaffolding. *CoRR*, abs/2509.06923, 2025. doi: 10.48550/ARXIV.2509.06923. URL https://doi.org/10.48550/arXiv.2509.06923.

Yong Lin, Hangyu Lin, Wei Xiong, and et al. Mitigating the alignment tax of RLHF. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 580–606. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.EMNLP-MAIN.35. URL https://doi.org/10.18653/v1/2024.emnlp-main.35.

Samuel Lippl, Thomas McGee, Kimberly Lopez, and et al. Algorithmic primitives and compositional geometry of reasoning in language models. *CoRR*, abs/2510.15987, 2025. doi: 10.48550/ARXIV.2510.15987. URL https://doi.org/10.48550/arXiv.2510.15987.

Bo Liu, Leon Guertler, Simon Yu, and et al. SPIRAL: self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *CoRR*, abs/2506.24119, 2025. doi: 10.48550/ARXIV.2506.24119. URL https://doi.org/10.48550/arXiv.2506.24119.

Shengcai Liu, Caishun Chen, Xinghua Qu, Ke Tang, and Yew-Soon Ong. Large language models as evolutionary optimizers. In *IEEE Congress on Evolutionary Computation, CEC 2024, Yokohama, Japan, June 30 - July 5, 2024*, pages 1–8. IEEE, 2024. doi: 10.1109/CEC60901.2024.10611913. URL https://doi.org/10.1109/CEC60901.2024.10611913.

Haipeng Luo, Qingfeng Sun, Can Xu, and et al. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025a. URL https://openreview.net/forum?id=mMPMHWOdOy.

Ziyang Luo, Can Xu, Pu Zhao, and et al. Wizardcoder: Empowering code large language models with evol-instruct. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=UnUwSIgK5W.

Ziyang Luo, Kaixin Li, Hongzhan Lin, and et al. Tree-of-evolution: Tree-structured instruction evolution for code generation in large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 297–316. Association for Computational Linguistics, 2025b. URL https://aclanthology.org/2025.acl-long.14/.

Ivan Moshkov, Darragh Hanley, Ivan Sorokin, and et al. AIMO-2 winning solution: Building state-of-the-art mathematical reasoning models with openmathreasoning dataset. *CoRR*, abs/2504.16891, 2025. doi: 10.48550/ARXIV.2504.16891. URL https://doi.org/10.48550/arXiv.2504.16891.

Alexander Novikov, Ngân Vu, Marvin Eisenberger, and et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *CoRR*, abs/2506.13131, 2025. doi: 10.48550/ARXIV.2506.13131. URL https://doi.org/10.48550/arXiv.2506.13131.

OpenAI. Openai GPT-5 system card. *CoRR*, abs/2601.03267, 2026. doi: 10.48550/ARXIV.2601.03267. URL https://doi.org/10.48550/arXiv.2601.03267.

Pedro Orvalho and Marta Kwiatkowska. Are large language models robust in understanding code against semantics-preserving mutations? *CoRR*, abs/2505.10443, 2025. doi: 10.48550/ARXIV.2505.10443. URL https://doi.org/10.48550/arXiv.2505.10443.

Long Phan, Alice Gatti, Ziwen Han, and et al. Humanity's last exam. *CoRR*, abs/2501.14249, 2025. doi: 10.48550/ARXIV.2501.14249. URL https://doi.org/10.48550/arXiv.2501.14249.

Steven T Piantadosi, Joshua B Tenenbaum, and Noah D Goodman. The logical primitives of thought: Empirical foundations for compositional cognitive models. *Psychological review*, 123(4):392, 2016.

Julien Pourcel, Cédric Colas, Gaia Molinaro, and et al. ACES: generating a diversity of challenging programming puzzles with autotelic generative models. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/7d0c6ff18f16797b92e77d7cc95b3c53-Abstract-Conference.html.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *CoRR*, abs/2201.02177, 2022. URL https://arxiv.org/abs/2201.02177.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, and et al. Mathematical discoveries from program search with large language models. *Nat.*, 625(7995):468–475, 2024. doi: 10.1038/S41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL http://arxiv.org/abs/1707.06347.

Vedant Shah, Dingli Yu, Kaifeng Lyu, and et al. Ai-assisted generation of difficult math questions. *CoRR*, abs/2407.21009, 2024. doi: 10.48550/ARXIV.2407.21009. URL https://doi.org/10.48550/arXiv.2407.21009.

Zhihong Shao, Peiyi Wang, Qihao Zhu, and et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL https://doi.org/10.48550/arXiv.2402.03300.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, and et al. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, pages 1279–1297. ACM, 2025. doi: 10.1145/3689031.3696075. URL https://doi.org/10.1145/3689031.3696075.

Quan Shi, Michael Tang, Karthik Narasimhan, and Shunyu Yao. Can language models solve olympiad programming? *CoRR*, abs/2404.10952, 2024. doi: 10.48550/ARXIV.2404.10952. URL https://doi.org/10.48550/arXiv.2404.10952.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, and et al. The curse of recursion: Training on generated data makes models forget. *CoRR*, abs/2305.17493, 2023. doi: 10.48550/ARXIV.2305.17493. URL https://doi.org/10.48550/arXiv.2305.17493.

Kate Smith-Miles and Leo Lopes. Measuring instance difficulty for combinatorial optimization problems. *Comput. Oper. Res.*, 39(5):875–889, 2012. doi: 10.1016/J.COR.2011.07.006. URL https://doi.org/10.1016/j.cor.2011.07.006.

Zafir Stojanovski, Oliver Stanley, Joe Sharratt, and et al. REASONING GYM: reasoning environments for reinforcement learning with verifiable rewards. *CoRR*, abs/2505.24760, 2025. doi: 10.48550/ARXIV.2505.24760. URL https://doi.org/10.48550/arXiv.2505.24760.

Anja Surina, Amin Mansouri, Lars Quaedvlieg, and et al. Algorithm discovery with llms: Evolutionary search meets reinforcement learning. *CoRR*, abs/2504.05108, 2025. doi: 10.48550/ARXIV.2504.05108. URL https://doi.org/10.48550/arXiv.2504.05108.

Gemma Team. Gemma 3 technical report. *CoRR*, abs/2503.19786, 2025a. doi: 10.48550/ARXIV.2503.19786. URL https://doi.org/10.48550/arXiv.2503.19786.

M-A-P Team. Supergpqa: Scaling LLM evaluation across 285 graduate disciplines. *CoRR*, abs/2502.14739, 2025b. doi: 10.48550/ARXIV.2502.14739. URL https://doi.org/10.48550/arXiv.2502.14739.

Qwen Team. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025c. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388.

Niki van Stein, Anna V. Kononova, Lars Kotthoff, and Thomas Bäck. Code evolution graphs: Understanding large language model driven design of algorithms. In Bogdan Filipic, editor, *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO 2025, NH Malaga Hotel, Malaga, Spain, July 14-18, 2025*, pages 943–951. ACM, 2025. doi: 10.1145/3712256.3726328. URL https://doi.org/10.1145/3712256.3726328.

Lev S Vygotsky. *Mind in society: The development of higher psychological processes*, volume 86. Harvard university press, 1978.

Haorui Wang, Marta Skreta, Cher Tian Ser, and et al. Efficient evolutionary search over chemical space with large language models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL https://openreview.net/forum?id=awWiNvQwf3.

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, and et al. Self-instruct: Aligning language models with self-generated instructions. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 13484–13508. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.754. URL https://doi.org/10.18653/v1/2023.acl-long.754.

Yubo Wang, Xueguang Ma, Ge Zhang, and et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract-Datasets_and_Benchmarks_Track.html.

Peng Xia, Kaide Zeng, Jiaqi Liu, and et al. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *CoRR*, abs/2511.16043, 2025. doi: 10.48550/ARXIV.2511.16043. URL https://doi.org/10.48550/arXiv.2511.16043.

Can Xu, Qingfeng Sun, Kai Zheng, and et al. Wizardlm: Empowering large pre-trained language models to follow complex instructions. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=CfXh93NDgH.

Qiying Yu, Zheng Zhang, Ruofei Zhu, and et al. DAPO: an open-source LLM reinforcement learning system at scale. *CoRR*, abs/2503.14476, 2025. doi: 10.48550/ARXIV.2503.14476. URL https://doi.org/10.48550/arXiv.2503.14476.

Zhiyuan Zeng, Yizhong Wang, Hannaneh Hajishirzi, and Pang Wei Koh. Evaltree: Profiling language model weaknesses via hierarchical capability trees. *CoRR*, abs/2503.08893, 2025. doi: 10.48550/ARXIV.2503.08893. URL https://doi.org/10.48550/arXiv.2503.08893.

Chong Zhang, Xiang Li, Jia Wang, and et al. Semantic-preserving adversarial attacks on llms: An adaptive greedy binary search approach. *CoRR*, abs/2506.18756, 2025a. doi: 10.48550/ARXIV.2506.18756. URL https://doi.org/10.48550/arXiv.2506.18756.

Kaiyan Zhang, Yuxin Zuo, Bingxiang He, and et al. A survey of reinforcement learning for large reasoning models. *CoRR*, abs/2509.08827, 2025b. doi: 10.48550/ARXIV.2509.08827. URL https://doi.org/10.48550/arXiv.2509.08827.

Andrew Zhao, Yiran Wu, Yang Yue, and et al. Absolute zero: Reinforced self-play reasoning with zero data. *CoRR*, abs/2505.03335, 2025. doi: 10.48550/ARXIV.2505.03335. URL https://doi.org/10.48550/arXiv.2505.03335.

Ruiyang Zhou, Shuozhe Li, Amy Zhang, and Liu Leqi. Expo: Unlocking hard reasoning with self-explanation-guided reinforcement learning. *CoRR*, abs/2507.02834, 2025. doi: 10.48550/ARXIV.2507.02834. URL https://doi.org/10.48550/arXiv.2507.02834.

Appendix for EvoTD

A Preliminaries

We leverage recent advances in RL techniques for LLM finetuning. We highlight two approaches that underpin our framework's design below.

A.1 Reinforcement Learning with Verifiable Rewards

RLVR is a training paradigm which fine-tunes a policy LLM π_θ using the reward that is collected from a deterministic verifier [Lambert et al., 2024]. The deterministic verifier $v : \mathcal{X} \to \{0, 1\}$ evaluates each model generation x_i against task-specific correctness criteria:

$$r_i = v(x_i) = \begin{cases} 1, & \text{if } x_i \text{ passes verification,} \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

The policy is optimized to maximize the expected verification reward over the data distribution. This binary feedback mechanism proves effective for structured reasoning domains where outcomes can be objectively validated through execution or formal checks. We adopt this verifier-based reward paradigm as the foundation for training our model.

A.2 Policy Optimization

Group Relative Policy Optimization (GRPO) GRPO [Shao et al., 2024] is a widely adopted policy gradient algorithm in RLVR that trains a policy π_θ without a separate, learned value function. It forms advantages by comparing multiple samples from the same prompt. Given a prompt x, sample a group $\{y_i\}_{i=1}^{G}$ from $\pi_{\theta_{\text{old}}}(\cdot \mid x)$ and compute scalar rewards $\{r_i\}_{i=1}^{G}$. GRPO normalizes rewards within the group to obtain a response-level advantage:

$$\hat{A}_i \;=\; \frac{r_i - \text{mean}(r_{1:G})}{\text{std}(r_{1:G}) + \epsilon_{\text{norm}}}. \tag{6}$$

Its policy updates follow a PPO [Schulman et al., 2017] style clipped surrogate objective with an explicit KL penalty to control policy drift:

$$\begin{aligned}\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{G}\sum_{i=1}^{G} \min\Big(\rho_i(\theta),\ \text{clip}(\rho_i(\theta), 1-\epsilon, 1+\epsilon)\Big) \\ \cdot \hat{A}_i + \beta\, \text{KL}(\pi_\theta(\cdot \mid x) \,\|\, \pi_{\theta_{\text{old}}}(\cdot \mid x))\end{aligned} \tag{7}$$

where $\rho_i(\theta) = \frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)}$. Maximizing $-\mathcal{L}_{\text{GRPO}}$ increases the likelihood of samples with positive group-relative advantages while keeping updates conservative.

Dynamic sAmpling Policy Optimization (DAPO) DAPO [Yu et al., 2025] refines the GRPO framework via algorithmic innovations. It uses an asymmetric clip-higher mechanism, removes the KL penalty, adopts token level policy gradient loss, uses dynamic sampling, and applies overlong reward shaping. The token-level advantages $\{\hat{A}_t^i\}$ are computed as in Equation 6. The objective averages clipped token-level updates across sampled outputs:

$$\mathcal{L}_{\text{DAPO}}(\theta) = \frac{1}{\sum_{i=1}^{G}|o^i|}\sum_{i=1}^{G}\sum_{t=1}^{|o^i|} \min\Big(r_t^i(\theta), \text{clip}(r_t^i(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}})\Big)\hat{A}_t^i \tag{8}$$

where $r_t^i(\theta) = \frac{\pi_\theta(o_t^i|q,o_{<t}^i)}{\pi_{\theta_{\text{old}}}(o_t^i|q,o_{<t}^i)}$ is the token-level probability ratio, subject to the constraint that $0 < |\{o^i \mid \text{is\_equivalent}(a, o^i)\}| < G$.

B Dataset and Benchmark Details

We provide detailed information about the benchmarks used in our experiments, including domain, variant type, and task count, complementing the high-level summary in Table 4. The selected benchmarks span rigorous code generation and competition-style symbolic reasoning. LiveCodeBench evaluates Python code generation under contamination-controlled conditions, while MBPP+ is a refactored variant of MBPP that increases difficulty through paraphrased prompts and stricter evaluation criteria. AIME'24, AIME'25, and OlympiadBench assess symbolic and algorithmic reasoning in competitive settings with rigorous correctness requirements and limited problem counts. To further probe broad multi-disciplinary knowledge and reasoning beyond code and math, we additionally evaluate on MMLU-Pro, which extends MMLU with reasoning-focused questions across 14 disciplines under a 10-option format, and SuperGPQA, which scales evaluation to 285 graduate-level disciplines spanning STEM, humanities, and professional fields.

C Additional Experiment Results

C.1 EvoTD Performance on General Domain Benchmarks

We further examine whether EvoTD's improvements transfer beyond code and math to broader reasoning benchmarks. EvoTD attains the highest average accuracy on both, lifting the Qwen3-4B

Table 4: Overview of benchmarks used for evaluation.

Dataset	Domain	# Tasks
LiveCodeBench^{v6} [Jain et al., 2025]	Code Generation (Python)	1055
MBPP+ [Austin et al., 2021]	Intro Programming (Python)	500
AIME'24 [AIME, 2025]	Symbolic Math (Mixed)	30
AIME'25 [AIME, 2025]	Symbolic Math (Mixed)	30
OlympiadBench [He et al., 2024]	Symbolic Math/Informatics	675
MMLU-Pro [Wang et al., 2024]	Multi-discipline Knowledge & Reasoning	12032
SuperGPQA [Team, 2025b]	Graduate-level Multi-discipline	26529

Table 5: Pass@1 Results on Thinking models across general domain reasoning benchmarks.

Domain / Model	Qwen3-4B	+Evol-Instruct	+EvoTD
Engineering	35.3	36.1	**37.6**
Medicine	36.0	**36.3**	35.9
Science	37.3	37.9	**40.6**
Philosophy	41.9	42.2	**42.7**
Military Science	35.6	35.2	**35.8**
Economics	43.8	43.7	**44.2**
Management	37.0	36.1	**37.1**
Sociology	32.4	32.3	33.2
Literature&Arts	**25.4**	25.1	25.3
History	22.4	**22.7**	22.3
Agronomy	33.6	**34.4**	33.5
Law	**36.2**	35.7	35.8
Education	**35.7**	35.4	35.2
AVG	35.5	36.0	**37.4**

(a) SuperGPQA

Domain / Model	Qwen3-4B	+Evol-Instruct	+EvoTD
Computer Science	71.1	74.0	**74.2**
Math	84.8	87.7	**88.9**
Chemistry	69.7	72.4	**75.3**
Engineering	44.0	45.2	**47.7**
Law	30.0	**32.8**	32.6
Biology	81.6	83.8	**84.1**
Health	62.7	**64.8**	64
Physics	69.9	72.6	**75.3**
Business	70.5	73.0	**74.3**
Philosophy	54.7	**55.9**	55.4
Economics	74.7	**58.5**	58.2
Other	56.5	**58.5**	58.2
Psychology	68.1	**69.0**	68.3
History	49.1	**51.8**	50.8
AVG	64.0	66.3	**67.1**

(b) MMLU-Pro

Thinking backbone from 64.0% to 67.1% on MMLU-Pro and from 35.5% to 37.4% on SuperGPQA, exceeding Evol-Instruct (66.3% and 36.0%) in each case. The per-domain pattern sharpens the interpretation: on MMLU-Pro, EvoTD's largest gains concentrate in STEM-adjacent domains in which abstract reasoning is the binding constraint — Chemistry (+5.6), Physics (+5.4), Math (+4.1), Engineering (+3.7), and Computer Science (+3.1) over the base. The same pattern recurs on SuperGPQA, where EvoTD's clearest gains arise on Science (+3.3) and Engineering (+2.3). These results establish that EvoTD's reasoning gains generalize cleanly beyond its in-domain training: the curriculum lifts overall Pass@1 on both general-domain benchmarks, with its largest improvements concentrated precisely where rigorous reasoning is the binding constraint.

C.2 Proposer Ablation

To evaluate EvoTD's sensitivity to the choice of proposer, we ablate our default configuration and evaluate the framework across a diverse suite of models, including both closed-source (o4-mini) and open-weight architectures (deepseek-r1-0528, gpt-oss-120B). We run the entire framework from scratch using each proposer, ranging from skill and complexity attribute extraction to task proposal. We then train Qwen3-4B Thinking under same amount (2,000) of synthesized tasks from each proposer. As shown in Table 6, performance gains remain consistent across proposers, with both Pass@1 and Pass@8 varying minimally across all evaluated benchmarks.

We attribute this proposer-agnostic stability to our multi-objective fitness check. Rather than relying on the proposer to generate flawless tasks on the first attempt, EvoTD regularizes the generated candidates through strict executability, skill alignment, and ZDP learnability checks. This ensures that only high-fidelity, structurally sound tasks enter the evolutionary population. Consequently, the framework neutralizes quality differences across LLM proposers, yielding a robust and scalable data synthesis pipeline that does not rely on the strongest closed-source models.

Table 6: Ablation study on proposer models across code and math benchmarks.

Proposer Model	LCB^{v6}	MBPP^{+}	AME24	AME25	Olympiad	**CAvg**	**MAvg**	**AVG**
Qwen3-4B Thinking Pass@1								
o4-mini	46.8	54.7	36.7	25.8	47.3	50.8	36.6	42.3
deepseek-r1-0528	47.5	55.1	37.3	23.8	48.3	51.3	36.5	42.4
gpt-oss-120B	46.7	55.2	37.4	25.1	48.2	51.0	36.9	42.5
Qwen3-4B Thinking Pass@8								
o4-mini	58.9	71.2	60.2	47.3	59.7	65.1	55.7	59.5
deepseek-r1-0528	59.5	70.1	54.7	44.2	60.7	64.8	53.2	57.8
gpt-oss-120B	58.8	71.4	56.3	48.4	61.5	65.1	55.4	59.3

D Additional Analysis

D.1 EvoTD's Sensitivity to Skill Bank

While we previously established the high fidelity and coverage of our extracted skill bank (Section D.2), a critical question remains: how sensitive is EvoTD to a degraded or incomplete initial ontology? To investigate this, we randomly remove 50% of the extracted skills (reducing the skill bank from 56 to 28) prior to the task generation phase. We then execute a single iteration of the evolutionary pipeline and train the Qwen3-4B Thinking model. As detailed in Table 7, this removal induces a uniform degradation in performance, lowering average Pass@1 by 1.1. This confirms that the framework's effectiveness is correlated with the breadth and quality of the initial skill bank. However, this ablation also highlights the robustness and sample efficiency of our evolutionary search mechanism. Even when bottlenecked by a constrained skill space, the single-iteration EvoTD achieves competitive performances with three iterations Evol-Instruct (average 41.2 Pass@1).

Table 7: EvoTD's sensitivity to initial skill bank for Qwen3-4B Thinking.

Method	LCB^{v6}	MBPP^{+}	AME24	AME25	Olympiad	**CAvg**	**MAvg**	**AVG**
Qwen3-4B Thinking Pass@1								
EvoTD (One Iteration)								
+Full Skills	46.8	54.7	36.7	25.8	47.3	50.8	36.6	42.3
+Half Skills	45.9	54.3	34.4	24.3	46.9	50.1	35.2	41.2
Evol-Instruct (Three Iteration)	47.4	53.7	35.4	22.4	46.9	50.6	34.9	41.2
Qwen3-4B Thinking Pass@8								
EvoTD (One Iteration)								
+Full Skills	58.9	70.9	60.2	47.3	59.7	64.9	55.7	59.4
+Half Skills	57.4	72.0	55.2	44.6	58.8	64.7	52.9	57.6
Evol-Instruct (Three Iteration)	58.0	54.8	55.0	44.2	59.1	56.4	52.8	54.2

D.2 Skill Coverage and Quality

As algorithmic skills are an important abstraction in our framework, it is critical to validate the quality and coverage of our extracted skill bank. Because there exists no universal gold standard for algorithmic skill ontologies, we validate our extraction by aligning it against the established, human-curated taxonomies from LeetCode[1] and Codeforces[2] (See full mappings in Table 8 and 9). As these official platforms frequently conflate canonical algorithmic concepts with broad or format-specific labels (e.g., "implementation", "interactive"), we report a more informative metric of reverse alignment: the proportion of our skills that successfully map back to recognized platform tags. **91.1%** of our extracted skills map directly to LeetCode, and **100.0%** map to Codeforces, confirming that the LLM extraction captures authentic reasoning primitives rather than hallucinating arbitrary concepts. Furthermore, our automated extraction demonstrates a **1.87**× and **1.47**× increase

[1] https://leetcode.com
[2] https://codeforces.com

in fine-grainedness over the LeetCode and Codeforces taxonomies, respectively. This fine-grained skill "resolution" (e.g. decomposing a coarse "Dynamic Programming" tag into precise skills like `tree_dp` or `bitmask_dp`) is a structural advantage. It yields the exact building blocks necessary for our `Skill Crossover` operator to compose novel reasoning tasks without relying on overly broad or ambiguous skills.

Table 8: Mapping from LeetCode tags to our skill bank.

LeetCode tag	**Our matching skills**
Array	prefix_sum, difference_array, range_sum_query
String	edit_distance, palindrome_expansion
Dynamic Programming	dp_1d, knapsack_dp, grid_dp, tree_dp, dp_on_dag, bitmask_dp, edit_distance, floyd_warshall
Math	gcd, integer_division, modular_arithmetic, sieve_of_eratosthenes, prime_factorization, divisor_enumeration, convex_hull, cross_product, polygon_area, rectangle_union_area, point_in_triangle, distance_computation
Two Pointers	two_pointers
Backtracking	brute_force_enumeration
Hash Table	hash_table
Linked List	—
Matrix	grid_dp
Recursion	—
Sorting	sorting, sweep_line
Binary Search	binary_search, parametric_search
Stack	—
Tree	tree_dp
Binary Tree	—
Bit Manipulation	prefix_xor, range_xor_query, bitmask_dp
Simulation	simulation
Depth-First Search	dfs, topological_sort
Greedy	interval_scheduling, graph_coloring_greedy, coupon_selection_greedy, fuel_station_greedy
Binary Search Tree	—
Sliding Window	sliding_window
Divide and Conquer	meet_in_the_middle
Monotonic Stack	—
Trie	—
Heap (Priority Queue)	priority_queue, dijkstra, prim
Merge Sort	—
String Matching	kmp_failure_function, rolling_hash
Combinatorics	binomial_coefficients, inclusion_exclusion
Memoization	—
Breadth-First Search	bfs, zero_one_bfs

Table 9: Mapping from official Codeforces tags to our skill bank.

Codeforces tag	**Our matching skills**
greedy	interval_scheduling, graph_coloring_greedy, coupon_selection_greedy, fuel_station_greedy
math	gcd, integer_division, modular_arithmetic, sieve_of_eratosthenes, prime_factorization, divisor_enumeration, binomial_coefficients
implementation	simulation
dp	dp_1d, knapsack_dp, grid_dp, tree_dp, dp_on_dag, bitmask_dp, edit_distance, floyd_warshall
constructive algorithms	—
data structures	prefix_sum, difference_array, range_sum_query, fenwick_tree, segment_tree, priority_queue, multiset
brute force	brute_force_enumeration
binary search	binary_search, parametric_search
sortings	sorting, sweep_line, mst_kruskal
graphs	bfs, topological_sort, prim, mst_kruskal
dfs and similar	dfs
trees	tree_dp
number theory	gcd, modular_arithmetic, sieve_of_eratosthenes, prime_factorization, divisor_enumeration
combinatorics	binomial_coefficients, inclusion_exclusion
strings	kmp_failure_function, rolling_hash, edit_distance, palindrome_expansion
bitmasks	bitmask_dp, prefix_xor, range_xor_query
two pointers	two_pointers, sliding_window
*special	—
geometry	convex_hull, cross_product, polygon_area, rectangle_union_area, point_in_triangle, distance_computation, sweep_line
dsu	union_find, mst_kruskal
divide and conquer	meet_in_the_middle
interactive	—
shortest paths	dijkstra, zero_one_bfs, floyd_warshall
games	—
probabilities	—
hashing	hash_table, rolling_hash
flows	—
matrices	grid_dp
fft	—
graph matchings	—
string suffix structures	—
ternary search	—
meet-in-the-middle	meet_in_the_middle
expression parsing	—
2-sat	—
chinese remainder theorem	—
schedules	interval_scheduling
communication	—

D.3 Crossover Task Fidelity

To verify that our `Skill Crossover` operator synthesizes genuinely synergistic tasks rather than superficial concatenations, we evaluate compositional fidelity using an LLM-as-a-Judge (`gpt-5-mini`). The judge rigorously audits structural interdependence (See prompt in Figure 4), explicitly rejecting tasks if they resemble isolated sub-problems, utilize skills decoratively, or include targeted skills that can be dropped without altering solvability. Under these strict criteria, **93%** (313 of 337) of the synthesized tasks are verified to possess deeply interconnected, non-trivial skill combinations. This confirms that our regularized crossover design effectively prevents combinatorial degeneration, faithfully yielding authentic, multi-skill reasoning tasks.

Compositional Synergy Evaluation Prompt

SYSTEM ROLE

You are an expert algorithm analyst. You answer every question with a single word: either Yes or No.

TASK OBJECTIVE

Your task is to determine whether the synthesized task meaningfully integrates all of the specified algorithmic skills.

INPUT VARIABLES

You will be given:

1. A synthesized task: `{task}`
2. A list of target algorithmic skills: `{skills}`
3. A description for each skill: `{skill_descriptions}`

EVALUATION RULES

- Answer "Yes" only if all listed skills are substantively involved in solving the task and interact in a unified way.
- Answer "No" if the task looks like a glued combination of skills rather than a coherent multi-skill problem.
- Answer "No" if any listed skill appears superficial, weakly involved, decorative, or droppable without substantially changing the task.
- Ignore superficial wording or explicit mentions of skills in the task description.
- Focus only on whether the core solving process genuinely requires the specified skills to work together.

OUTPUT FORMAT

Does this task meaningfully integrate all of the specified skills?
Respond with only one word:
`Yes` or `No`

Figure 4: Compositional Synergy Evaluation Prompt

D.4 Target Skill and Actual Solution Alignment

One critical aspect of targeted data synthesis involves aligning the solver's behavior with the intended task logic, ensuring it employs the targeted algorithmic skills rather than exploiting alternative heuristics. Within EvoTD, this solver-skill alignment is structurally guaranteed for Deduction (T_{ded}) and Abduction (T_{abd}) tasks, as the model reasons directly over provided code snippets where the targeted logic is already explicitly embedded. Induction tasks (T_{ind}) require *de novo* code generation and are thus theoretically susceptible to alternative solution paths. However, their problem statement and I/O pairs are strictly derived from verified reference implementations, establishing a strong prior toward the intended logic. To empirically quantify this consistency, we employed an LLM-as-a-Judge (`gpt-5-mini`) to audit correct solver rollouts for Induction tasks (See prompt in Figure 5). **87.4%** (1,371 of 1,568) of successful solutions authentically implementing the targeted skills, confirming the robust target skill and solution alignment.

Solver-Skill Alignment Evaluation Prompt

SYSTEM ROLE

You are an expert algorithm analyst. You answer every question with a single word: either Yes or No.

TASK OBJECTIVE

Your task is to determine whether the code meaningfully implements or applies all of the specified skills.

INPUT VARIABLES

You will be given:

1. A Python code snippet: `{code}`
2. A list of algorithmic skills: `{skills}`
3. A description for each skill: `{skill_descriptions}`

EVALUATION RULES

- Answer "Yes" only if every listed skill is genuinely used in the code's core algorithmic logic.
- Answer "No" if even one listed skill is not meaningfully applied.
- Answer "No" if the code uses a different approach, even if the final output is correct.
- Ignore helper boilerplate such as input/output handling, type annotations, wrappers, or formatting details.
- Focus only on whether the essential reasoning or computation in the code depends on all specified skills.

OUTPUT FORMAT

Respond with only one word:
`Yes` or `No`

Figure 5: Solver-Skill Alignment Evaluation Prompt

D.5 Evolutionary Operators Ablation

Table 10 presents the fine-grained performance breakdown across individual benchmarks for the ablation study. This detailed view reveals distinct functional roles for each operator:

- **Attribute Mutation for Hard Code & Math:** Removing `Attribute Mutation` causes the most significant drops in LiveCodeBench (-1.0%) and AIME 24 (-5.0%). In contrast, MBPP+ (a simpler coding benchmark less reliant on complex reasoning) remains largely unaffected. This is expected: tasks requiring multi-step complex reasoning benefit most from fine-grained difficulty scaling, which is precisely what `Attribute Mutation` provides.
- **Skill Crossover for Reasoning Transfer:** While the model without `Skill Crossover` maintains strong performance on standard coding tasks (e.g., MBPP+), it suffers on abstract reasoning benchmarks. This suggests that while simple coding skills can be learned via isolation, high-level mathematical reasoning (AIME/Olympiad) requires the diverse combinatorial exposure.

Table 10: Detailed performance breakdown on effectiveness of Attribute Mutation and Skill Crossover

Model	LCB^{v6}	MBPP^{+}	AME24	AME25	Olympiad	CAvg	MAvg	AVG
Qwen3-4B Thinking *Pass@1*								
EvoTD	**49.8**	55.8	**42.2**	**29.0**	**49.4**	52.8	**40.2**	**45.2**
w/o Attribute Mutation	48.8	56.5	37.2	28.6	48.6	52.7	38.1	43.9
w/o Skill Crossover	49.1	**57.0**	39.1	27.4	49.2	**53.1**	38.6	44.4
Qwen3-4B Thinking *Pass@8*								
EvoTD	**61.5**	71.2	**63.2**	49.9	62.1	66.4	**58.4**	**61.6**
w/o Attribute Mutation	60.9	71.7	60.9	48.7	60.1	66.3	56.6	60.5
w/o Skill Crossover	60.5	**73.8**	59.2	**50.0**	**62.2**	**67.2**	57.1	61.1

E Computing Infrastructure

We conduct our experiments on a server equipped with 4 NVIDIA H200 (141GB VRAM) GPUs. We utilize the NVIDIA CUDA toolkit version 13.0. All experiments are implemented using Python 3.10.19, the PyTorch framework version 2.9.0, and the Transformer library version 4.57.3.

F Cost Analysis

Latency To quantify the operational overhead of our framework, we analyze the end-to-end wall-clock compute breakdown for both the Qwen3-4B Base and Qwen3-4B Thinking models (Table 11). This profiling demonstrates that our evolutionary data synthesis process (encompassing task proposal, skill alignment, and the learnability check) is highly efficient, accounting for only 13.5% of the total runtime for the Thinking model and 16.1% for the Base model.

API Cost The API cost for o4-mini, gpt-oss-120b, and deepseek-r1-0528 per iteration synthesis is \$100, \$120, and \$110, respectively.

Proposal Efficiency To ensure a fair comparison, we standardize the training set size for both EvoTD and all baselines at approximately 6,000 samples across three iterations. In each iteration, EvoTD generates roughly 3,142 candidates to yield 2,000 valid samples after multi-objective verification (`discard_rate`=36.3%). In contrast, Evol-Instruct requires approximately 5,463 candidates to secure the same volume of valid data (`discard_rate`=63.4%). This demonstrates that EvoTD is a significantly more sample-efficient method for synthesizing diverse curricula.

Table 11: Wall-clock compute breakdown for EvoTD.

Component	Qwen3-4B Thinking	Fraction	Qwen3-4B Base	Fraction
Task Proposal	1.6 hr	2.8%	1.6 hr	6.1%
Skill Alignment	0.7 hr	1.2%	0.7 hr	2.7%
Learnability	5.5 hr	9.5%	1.9 hr	7.3%
RLVR Training	49.9 hr	86.5%	21.9 hr	83.9%

G Implementation Details

Hyperparameters Table 13 and 12 present the training hyperparameters we use in Base model and Thinking model RL training, respectively. Table 15 and 14 show the evaluation hyperparameter configuration for Base and Thinking models.

Table 12: Qwen3-4B Thinking Training Hyperparameters

Parameter	Value
Model Configuration	
Max Prompt Length	6144
Max Response Length	8096
Seed Batch Factor	42
Training Settings	
Train Batch Size	64
Generation Batch Size	88
Learning Rate	1e-6
Optimizer	AdamW
Weight Decay	0.01
Grad Clip	1.0
Total Steps	200
RL Settings	
Adv. Estimator	DAPO
KL Loss	0.0
KL Reward	0.0
Entropy Coefficient	0.0
PPO Epochs	16
N Rollouts	16
Rollout Temperature	1.0
Rollout Top-P	0.99
N Samples to Estimate Task Accuracy	10
Overlong Penalty Factor	1.0
Overlong Penalty Buffer	1024
Dynamic Filtering	True
Max Number Generation Batch	10

Table 13: Qwen3-4/8B Base, LLaMA-3.2-3B/3.1-8B Instruct Training Hyperparameters

Parameter	Value
Model Configuration	
Max Prompt Length	6144
Max Response Length	8096
Seed Batch Factor	42
Training Settings	
Train Batch Size	64
Generation Batch Size	88
Learning Rate	1e-6
Optimizer	AdamW
Weight Decay	0.01
Grad Clip	1.0
Total Steps	200
RL Settings	
Adv. Estimator	DAPO
KL Loss	0.0
KL Reward	0.0
Entropy Coefficient	0.0
PPO Epochs	16
N Rollouts	16
Rollout Temperature	1.0
Rollout Top-P	0.99
N Samples to Estimate Task Accuracy	10
Overlong Penalty Factor	0.0
Overlong Penalty Buffer	0
Dynamic Filtering	True
Max Number Generation Batch	10

Table 14: Qwen3-4B Thinking Evaluation Hyperparameters

Parameter	Value
Max Response Length	8096
Temperature	0.6
Top-P	0.95
Number of Samples	8

Table 15: Qwen3-4/8B Base, LLaMA-3.2-3B/3.1-8B Instruct Evaluation Hyperparameters

Parameter	Value
Max Response Length	8096
Temperature	0.6
Top-P	0.95
Number of Samples	8

H Qualitative Examples

We present qualitative examples for our `Attribute Mutation` and `Skill Crossover` in Figure 6 and Figure 7, respectively.

`Attribute Mutation` This mutation introduces a secondary sliding window phase that processes the output of the first window (the sums) rather than the raw input. Specifically, it applies a monotonic queue (deque) to efficiently find the maximum value within a second window of size m over the intermediate sums, increasing the algorithmic complexity from a simple aggregation to a multi-stage dual-window pipeline.

Mutation: Window Size

```python
from collections import deque

def f(arr: list[int], k: int, m: int):
    n = len(arr)
    if k <= 0 or k > n:
        return -1
    sums = []
    window_sum = sum(arr[:k])
    sums.append(window_sum)
    for i in range(k, n):
        window_sum += arr[i] - arr[i - k]
        sums.append(window_sum)
    # === MUTATION START: Max-Window Phase ===
    m1 = len(sums)
    if m <= 0 or m > m1:
        return -1
    maxes = []
    dq = deque()
    for i in range(m1):
        while dq and sums[dq[-1]] <= sums[i]:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - m:
            dq.popleft()
        if i >= m - 1:
            maxes.append(sums[dq[0]])
    # === MUTATION END ===

    L = min(len(sums), len(maxes))
    diffs = [maxes[i] - sums[i] for i in range(L)]
    min_diff = diffs[0]
    idx = 0
    for i in range(1, L):
        if diffs[i] < min_diff:
            min_diff = diffs[i]
            idx = i
    return idx
```

Figure 6: Illustration of Attribute Mutation, showing how complexity is increased through an additional max-window operation without altering the core algorithmic logic.

`Skill Crossover` The code integrates **parametric search** to optimize the number of updates, using a **binary search** framework to verify feasibility. Inside each verification step, it employs a **difference array** combined with prefix sums to efficiently reconstruct the polygon's y-coordinates in linear time, before finally applying the shoelace formula to calculate the **polygon area**. This layering allows for determining the minimal prefix of updates required to meet the area target without simulating the entire process quadratically.

Multi-Skill Crossover Analysis

```python
def f(n, updates, target):
    xs = list(range(n))

    # Skill: Polygon Area (Shoelace Formula)
    def compute_area(ys):
        area = 0
        for i in range(n):
            j = (i + 1) % n
            area += xs[i] * ys[j] - xs[j] * ys[i]
        return abs(area) / 2

    def area_for(k):
        diff = [0] * (n + 1)

        # Skill: Difference Array (Range Updates)
        for i in range(k):
            (l, r, delta) = updates[i]
            diff[l] += delta
            diff[r + 1] -= delta

        ys = [0] * n
        acc = 0

        # Skill: Prefix Sum (Reconstruction)
        for i in range(n):
            acc += diff[i]
            ys[i] = acc
        return compute_area(ys)

    (left, right) = (1, len(updates))
    ans = -1

    # Skill: Parametric Search (Binary Search on Answer)
    while left <= right:
        mid = (left + right) // 2
        if area_for(mid) >= target:
            ans = mid
            right = mid - 1
        else:
            left = mid + 1
    return ans
```

Figure 7: Example code integrating parametric search, difference array, prefix sum, and polygon area.

I Skills and Attributes

Figures 8 and 9 present the skills and complexity attributes we collect from USACO Shi et al. [2024]. Each skill and attribute are associated with descriptions that help the proposer better understand the meaning.

Skill Bank

Sorting & Searching

- **sorting**: Arrange elements in non-decreasing order using a comparison-based sort in $O(nlogn)$ time.
- **binary search**: Locate a target or insertion index in a sorted array in $O(logn)$ time.
- **two pointers**: Use two indices to traverse or maintain a window over a sequence under constraints in $O(n)$ time.
- **sliding window**: Maintain a variable or fixed-size window on a sequence, updating aggregate statistics in $O(n)$ time.
- **parametric search**: Perform binary search over the answer space using a monotonic feasibility predicate.

Prefix & Difference Arrays

- **prefix sum**: Compute cumulative sums of an array to answer range-sum queries in $O(1)$ or $O(logn)$ time.
- **difference array**: Use boundary differences to support range-add updates in $O(1)$ and reconstruct values via prefix sum.
- **range sum query**: Query the sum of any subarray using precomputed prefix sums or Fenwick tree in $O(1)$ or $O(logn)$.
- **prefix xor**: Compute cumulative XORs of an array to enable subarray XOR queries in $O(1)$.
- **range xor query**: Answer the XOR of any contiguous subarray by combining two prefix XOR values in $O(1)$.

Data Structures

- **union find**: Maintain disjoint sets with union-by-rank and path compression, supporting near-constant-time union and find.
- **fenwick tree**: Support point updates and prefix-sum queries on an array in $O(logn)$ time.
- **segment tree**: Support point and range queries or updates on an array in $O(logn)$ time, with optional lazy propagation.
- **hash table**: Map keys to values with average $O(1)$ insert, delete, and lookup operations.
- **priority queue**: Retrieve and remove the minimum or maximum element in $O(logn)$ time.
- **multiset**: Maintain a multiset allowing duplicates with efficient insertions, deletions, and order queries.

Graph & Trees

- **bfs**: Traverse or compute distances in unweighted graphs level by level using a queue in $O(V + E)$.
- **dfs**: Recursively or iteratively traverse a graph depth-first to explore all reachable nodes in $O(V + E)$.
- **dijkstra**: Compute shortest paths in graphs with non-negative weights using a priority queue in $O((V + E)$ log V).
- **zero one bfs**: Compute shortest paths in graphs with 0/1 edge weights using a deque in $O(V + E)$.
- **floyd warshall**: Compute all-pairs shortest paths in a weighted graph in $O(n^3)$ time.
- **topological sort**: Order nodes of a DAG so that all edges go from earlier to later in $O(V + E)$ time.
- **mst kruskal**: Construct a minimum spanning tree by sorting edges and using union-find in $O(ElogE)$ time.
- **prim**: Build a minimum spanning tree by growing from a start node with a priority queue in $O(ElogV)$.

Dynamic Programming

- **dp 1d**: Use a one-dimensional array to iteratively compute optimal values with local transitions in $O(n)$ time.
- **knapsack dp**: Solve the 0/1 knapsack problem by updating a capacity-indexed DP array in $O(nC)$ time.
- **grid dp**: Perform DP on a 2D grid with state transitions from neighboring cells in $O(nm)$ time.
- **tree dp**: Aggregate child results at each node in a tree via post-order traversal in $O(n)$ time.
- **dp on dag**: Compute DP on a DAG by processing nodes in topological order and relaxing outgoing edges.
- **bitmask dp**: Perform DP over subsets represented by bitmasks, iterating through masks for combinatorial counts.

Math & Number Theory

- **gcd**: Compute the greatest common divisor of two integers via the Euclidean algorithm in $O(logmin(a, b))$.
- **integer division**: Perform floor and ceiling division on integers in $O(1)$ time.
- **modular arithmetic**: Perform addition, subtraction, multiplication, and exponentiation under a modulus.
- **sieve of eratosthenes**: Precompute primality of numbers up to N in $O(NloglogN)$ time.
- **prime factorization**: Factor an integer into primes by trial division up to its square root in $O(\sqrt{n})$ time.
- **divisor enumeration**: Enumerate all divisors of an integer by scanning up to its square root in $O(\sqrt{n})$ time.
- **binomial coefficients**: Precompute factorials and inverse factorials mod a prime to answer n choose k queries in $O(1)$.

Geometry & Computational Geometry

- **convex hull**: Compute the convex hull of a point set in $O(nlogn)$ time using Graham scan or monotonic chain.
- **cross product**: Compute the 2D cross product to test orientation or compute signed area contributions.
- **polygon area**: Compute the area of a simple polygon via the shoelace formula in $O(n)$ time.

- **rectangle union area**: Compute union area of axis-aligned rectangles with a sweep-line and segment coverage in $O(nlogn)$.
- **point in triangle**: Test if a point lies inside a triangle via area or barycentric coordinate checks in $O(1)$.
- **distance computation**: Compute Euclidean or Manhattan distance between points in $O(1)$ time.

String Algorithms

- **kmp failure function**: Compute the prefix-function for a pattern in $O(m)$ to support efficient substring search.
- **rolling hash**: Compute hash fingerprints of strings or substrings for $O(1)$ comparison with precomputed powers.
- **edit distance**: Compute the minimum cost of insertions, deletions, and substitutions between two strings via DP in $O(nm)$.
- **palindrome expansion**: Enumerate palindromic substrings by expanding around centers in $O(n^2)$ time (or use Manacher's for $O(n)$).

Greedy Algorithms

- **interval scheduling**: Select the maximum number of non-overlapping intervals by sorting by end time and greedy selection in $O(nlogn)$.
- **graph coloring greedy**: Assign colors to graph vertices in a fixed order by choosing the smallest available color per vertex.
- **coupon selection greedy**: Under a budget constraint, iteratively pick the locally optimal purchase option to maximize items acquired.
- **fuel station greedy**: Minimize fuel cost along a route by computing next cheaper station using a monotonic stack in $O(n)$.

Miscellaneous Techniques

- **meet in the middle**: Split a problem in half, compute results on each side, and combine in $O(2^{n/2})$ time.
- **brute force enumeration**: Systematically iterate over all candidates in a finite search space to test each solution.
- **sweep line**: Process events in sorted order while maintaining active segments or counts in a data structure.
- **inclusion exclusion**: Count elements satisfying at least one of several conditions by alternately adding and subtracting intersection counts.
- **simulation**: Model and execute step-by-step state changes according to rules to obtain final outcomes.

Figure 8: Skill Categories and Descriptions

Complexity Attribute List

input_size_n: Primary scale N of the input data (e.g., number of elements, string length, graph nodes); directly influences time and space complexity.

test_case_count: Number of independent test cases T; total runtime and memory scale linearly with T.

query_count: Number of queries Q or subproblem invocations; overall work typically multiplies by Q.

update_count: Number of update or modification operations U; more updates increase total processing time accordingly.

alphabet_size: Number of distinct symbols or characters σ; affects automaton construction, transition branching, and memory for alphabet-indexed structures.

value_range: Range or domain size of numeric input values; large ranges may require coordinate compression or affect data-structure performance.

bit_length: Bit-length of integer values; larger magnitudes increase the cost of arithmetic operations like multiplication, division, and modulo.

graph_vertices: Number of vertices N in a graph; determines sizes of adjacency lists/matrices and base complexity of graph algorithms.

graph_edges: Number of edges M in a graph; influences traversal and update costs by scaling adjacency operations.

directed_graph: Flag indicating whether a graph is directed; directed edges alter adjacency considerations and may change algorithmic choices.

dynamic_graph: Flag for dynamic graph updates (insertions/removals); dynamic scenarios require specialized data structures and often incur extra log- or amortized costs.

pattern_length: Length m of a pattern or substring; affects DP dimensions, automaton states, and preprocessing time, typically linearly in m.

window_size: Size k of a sliding window or local segment; controls how many elements are processed per window operation.

segment_count: Number of segments or intervals N (e.g., line segments, time intervals); larger counts raise event-based sweep or segment-tree operation costs.

dp_state_dimensions: Number of independent parameters tracked in a DP state; each additional dimension multiplies the DP table and transition work.

dp_transition_branching: Average number of transitions or options per DP state; higher branching factors increase per-state transition complexity.

search_space_size: Total size of the combinatorial search space; directly bounds exhaustive or backtracking algorithm runtimes.

branching_factor: Branching factor in recursive or enumeration algorithms; high factors lead to exponential state-space growth.

recursion_depth: Maximum depth of recursion or number of iterative steps; deeper recursion increases total computation and call-stack usage.

monotonicity: Boolean indicating whether a predicate or metric behaves monotonically, enabling search optimizations like two-pointers or binary search.

grid_dimensions: Dimensions (rows, columns) of a grid or matrix; larger grids raise state counts and traversal costs, often quadratically in size.

coordinate_dimension: Spatial dimensionality D of geometric inputs; higher dimensions expand neighbor counts and combinatorial complexity.

constraint_count: Number of constraints C (e.g., inequalities, logic rules); more constraints typically increase checking and preprocessing time.

constraint_type_count: Number of distinct constraint forms or operators; more types enlarge case-handling logic and may raise constant factors.

operation_type_count: Number of allowed operation types (e.g., insertion, deletion, substitution); more types expand DP branching and search transitions.

operation_cost_variability: Degree of variability in operation costs; nonuniform costs require weighted algorithms or more complex DP logic.

objectives_count: Number of simultaneous objectives to optimize; multi-objective problems often require vector comparisons and higher-dimensional reasoning.

Figure 9: Complexity Attribute List

J Limitations

Dependence on the initial skill bank. The effectiveness of EvoTD is partly contingent on the quality and breadth of the initial skill bank. We mitigate this through a data-driven extraction pipeline that leverages LLM metacognition (§2.2) and verify robustness to substantial taxonomy reduction in Appendix D.1, where halving the skill bank still yields competitive performance. Nevertheless, gains may attenuate in domains where authoritative skill primitives are sparse or poorly structured.

Reliance on programmatically verifiable rewards. Our multi-objective fitness check is naturally instantiated in domains that admit deterministic verification—algorithmic code and symbolic mathematics in this work. Extension to open-ended reasoning settings without programmatic oracles (e.g., long-form argumentation, multimodal grounding) would require substituting the executor with alternative validators such as learned reward models, LLM-as-judge protocols, or finer-grained rubrics rewards. We view this as a natural avenue for future investigation rather than an obstacle to the underlying methodology.

Static skill and complexity-attribute taxonomies. Both the skill bank $\mathcal{S}$ and the complexity attribute set $\mathcal{C}$ are extracted once from the seed corpus and held fixed throughout the evolutionary process. Although our quality and coverage analysis (Appendix D.2) confirms the extracted bank is already comprehensive, finer-grained primitives or emergent skill compositions that arise as the solver matures are not automatically incorporated. Online taxonomy expansion, where new skills are discovered or refined during training, is a promising direction we leave to subsequent work.

K EvoTD Algorithm

Algorithm 1: EvoTD Algorithm

Require: Proposer π_{prop}; Solver π_{sol}; Verifier $\mathcal{V}$; Seed $\mathcal{D}_{\text{seed}}$
Ensure: Training Tasks $\mathcal{D}_{\text{final}}$

$\mathcal{S} \leftarrow \text{ExtractSkills}(\pi_{\text{prop}}, \mathcal{D}_{\text{seed}})$, $\mathcal{D}^{\text{ind}} \leftarrow \emptyset$, $\mathcal{C} \leftarrow \text{ExtractAttributes}(\pi_{\text{prop}}, \mathcal{D}_{\text{seed}})$, $\mathcal{D}_{\text{final}} \leftarrow \emptyset$
for each type $\in \{\text{Ded}, \text{Abd}\}$ **do**
  $\mathcal{D}^{\text{type}}_{\text{skill}} \leftarrow \emptyset$, $\mathcal{D}^{\text{type}}_{\text{mutate}} \leftarrow \emptyset$, $\mathcal{D}^{\text{type}}_{\text{crossover}} \leftarrow \emptyset$
  ▷ *Seeding: Skill-conditioned Task Synthesis*
  while $\mathcal{S} \neq \emptyset$ **do**
    $t_s^{\text{type}} \leftarrow \text{SkillSeeding}(\pi_{\text{prop}}, s, \text{type})$
    if $\mathcal{V}(t_s^{\text{type}}, \pi_{\text{sol}}, s)$ **then**
      $\mathcal{D}^{\text{type}}_{\text{skill}}.\text{append}(t_s^{\text{type}})$, $\mathcal{S}.\text{remove}(s)$
    end if
  end while
  ▷ *Mutation: Mutate over Applicable Attributes*
  while $\mathcal{S} \neq \emptyset$ **do**
    $t_s^{\text{type}} \leftarrow \mathcal{D}^{\text{type}}_{\text{skill}}[s]$, Mutated $\leftarrow$ False
    while $n \neq 0$ **do**
      $t_s^{\text{type}'} \leftarrow \mathcal{M}_{\text{attr}}(t_s^{\text{type}}, \pi_{\text{prop}})$
      if $\mathcal{V}(t_s^{\text{type}'}, \pi_{\text{sol}}, s)$ **then**
        $\mathcal{D}^{\text{type}}_{\text{mutate}}.\text{append}(t_s^{\text{type}'})$, Mutated $\leftarrow$ True
      end if
      if Mutated $=$ True **then**
        $\mathcal{S}.\text{remove}(s)$
      end if
    end while
  end while
  ▷ *Crossover: Explore Novel Skill Combinations*
  while $\mathcal{S} \neq \emptyset$ **do**
    $t_s^{\text{type,new}} \leftarrow \mathcal{X}_{\text{skill}}\left(\Lambda\left(\mathcal{D}^{\text{type}}_{\text{skill}} \cup \mathcal{D}^{\text{type}}_{\text{mutate}}\right), \pi_{\text{prop}}\right)$, Crossovered $\leftarrow$ False
    if $\mathcal{V}(t_s^{\text{type,new}}, \pi_{\text{sol}}, s)$ **then**
      $\mathcal{D}^{\text{type}}_{\text{crossover}}.\text{append}(t_s^{\text{new}})$, Crossovered $\leftarrow$ True
    end if
    if Crossovered $=$ True **then**
      $\mathcal{S}.\text{remove}(s)$
    end if
  end while
  $\mathcal{D}_{\text{final}} \leftarrow \mathcal{D}^{\text{type}}_{\text{skill}} \cup \mathcal{D}^{\text{type}}_{\text{mutate}} \cup \mathcal{D}^{\text{type}}_{\text{crossover}}$
end for
▷ *Synthesize Ind. Task from Ded. and Abd. Task*
for $t \in \mathcal{D}_{\text{final}}$ **do**
  $t^{\text{ind}} \leftarrow \text{Synthesis}(\pi_{\text{prop}}, t)$
  if $\mathcal{V}(t^{\text{ind}}, \pi_{\text{sol}})$ **then**
    $\mathcal{D}^{\text{ind}}.\text{append}(t^{\text{ind}})$
  end if
end for
return $\mathcal{D}_{\text{final}} \cup \mathcal{D}^{\text{ind}}$

L Prompts

We organize proposer synthesis prompts by their functional role in EvoTD.

Skill and Attribute. Figures 10, 11, 12, and 13 correspond to prompts used for extracting atomic skills and attributes, clustering skills and attributes, and reflecting on skill usage.

Abduction Tasks. We list prompts for skill-based abduction task synthesizing (Figures 14), mutating abduction tasks (Figures 15), crossovering abduction tasks (Figures 16). Figure 17 shows the prompt for the solver to predict the synthesized abduction tasks.

Deduction Tasks. Similar to Abduction tasks, we present prompts for skill-based deduction task synthesizing (Figures 18), mutating deduction tasks (Figures 19), crossovering deduction tasks (Figures 20). Figure 21 shows the prompt for the solver to predict the synthesized deduction tasks.

Induction Tasks. Figures 22 present induction task prompts. Inspired by Li et al. [2025], Zhou et al. [2025], we ask the proposer to generate leetcode style hints to difficult induction tasks (solver *pass@10*=0) (Figures 23) Figures 24 shows the prompt for the solver to predict the synthesized induction tasks.

Skill Attribute Prompt

You are an expert Computer Science professor and a seasoned competitive programming coach.
Your task is to analyze the provided programming problem and its reference solutions to identify:

1. The **ATOMIC** algorithmic skills needed to solve the problem
2. The **complexity attributes** – all characteristics that affect the problem's complexity/difficulty or could be varied to create mutations

Your analysis must be precise, accurate, and adhere to the standardized terminology of the field.

PART 1: SKILL IDENTIFICATION

1. **Holistic Analysis**: You must consider both the problem statement and the provided solutions.
   - The problem statement provides context and hints at the required complexity.
   - The solution code provides the definitive implementation, revealing the exact algorithms skills used.
   - If multiple solutions exist, include the union of distinct skills they use. Do not infer skills not evidenced by the code.
2. **Skills**
   - Identify the single most important and necessary concept that the problem is testing. Then, break down the concept into secondary or sub-level techniques/skills of the concept required to solve the problem.
   - Report only algorithmic programming-level techniques/data-structure operations (e.g., binary_search, two_pointers, monotonic_stack, dijkstra, fenwick_tree, prefix_sum, etc.).
   - Avoid overly generic terms (e.g., "logic," "math") or trivial implementation details (e.g., "variables," "loops").
   - Skills must be ATOMIC: a single, independent technique—not a composite or pipeline.
     - Split composites into primitives (e.g., segment_tree_with_lazy_propagation → segment_tree, lazy_propagation; binary_search_on_answer → parametric_search; etc.).
   - Names must be **lower_snake_case**, descriptive, and canonical (no synonyms, no duplicates).
   - Provide a brief, precise description of the skill.

PART 2: COMPLEXITY ATTRIBUTES

Analyze the solution and identify **every attribute that contributes to the problem's complexity or could be modified to create variations**. Think like a problem setter who needs to create variants of the problem with varying complexities.

Discovery Process:

For each major component of the solution (input/output, loops, data structures, operations, conditions, etc.), ask:

- What choices were made here?
- What constraints or bounds exist?
- What could be different while maintaining the core algorithm (skill)?

Categories to Explore (non-exhaustive):

Quantitative Attributes - Numeric values that affect complexity:

- Input size constraints (array length, string length, matrix dimensions)
- Value ranges (minimum/maximum values for integers, characters used)
- Iteration bounds (loop limits, recursion depth)
- Precision requirements (decimal places, modulo values)
- Count limits (number of operations allowed, query limits)

Structural Attributes - How the solution is organized:

- Data structure choices (array vs linked list, set vs map, etc.)
- Traversal patterns (left-to-right, inside-out, breadth-first vs depth-first, etc.)
- Processing order (sorted vs unsorted, online vs offline, etc.)
- Storage strategy (in-place vs auxiliary space, etc.)

Operational Attributes - What operations are performed:

- Core operations (addition vs multiplication, min vs max, etc.)
- Comparison types (strict vs non-strict inequality, etc.)
- Combination methods (sum, product, XOR, concatenation)
- Update strategies (increment, replace, accumulate, etc.)

Logical Attributes - Decision-making patterns:

- Branching conditions (what triggers different paths, etc.)
- Early termination conditions (when to stop)
- Selection criteria (how elements are chosen, etc.)

- Validation rules (what makes a solution valid)

Complexity Factors - What makes this instance hard:

- Number of nested structures
- Dependencies between computations
- State tracking requirements
- Edge case handling needs

Extraction Guidelines:

For each complexity attribute you identify, provide:

1. **Attribute name**: A descriptive **lower_snake_case** identifier
2. **Description**: A clear definition of what this attribute represents. How varying this attribute affects problem difficulty.

PART 3: OUTPUT FORMAT

Your final response **must be valid JSON** exactly matching the schema below. Please output **ONLY** the JSON object. Do not include any extra things.

```
{"skills": {
    <name of the skill>: <description of the skill>,
    ...
  },
  "attributes": {
    <name of the attribute>: <description of the attribute>,
    ...
  }}
```

Input

Problem:
{problem}
Reference Code Solution:
{code_solution}

Figure 10: Skill Attribute Prompt

Cluster Skill Prompt

You are an expert computer science curriculum designer with extensive experience in creating structured learning paths for algorithmic problem-solving.

Your goal is to take a large, potentially redundant list of skills and perform a two-step refinement process:

1. **Deduplicate**: Merge overlapping or similar skills into a set of canonical, non-overlapping skills with clear, consolidated descriptions.
2. **Categorize**: Group the resulting canonical skills into broader categories based on primary algorithmic concepts or data-structure families.

INPUT

You will be given a list of skills, where each skill has a `skill` name and a `description`.
`{skill_list}`

YOUR TASK

Step 1: Deduplicate into Canonical Skills (Mental Step)

- First, mentally merge all synonymous, overlapping, or related skills from the input list into a single, canonical skill.
- If a skill from the input list is already distinct and does not overlap with any others, treat it as a canonical skill on its own; no deduplication is needed for it.
- For each canonical skill, define a standard `skill` name (in `lower_snake_case`) and write a new concise `description` that synthesizes the core concept. For example, `array_sorting` and `custom_sorting_logic` should be merged into a single canonical skill called `sorting`.

Step 2: Group Canonical Skills into Categories

- Now, take the set of canonical skills you just defined.
- Group these skills into high-level categories (e.g., `graph_algorithms`, `dynamic_programming`, `greedy_algorithms`).
- The final output should be a list of these categories, each containing the relevant canonical skills you created.

OUTPUT

Your final response **must be a list of valid JSON objects** exactly matching the schema below.
Please output **ONLY** the JSON. Do not include any extra things.
List of JSON SCHEMA TO FOLLOW:

```
{
  "category": <name of the category>,
  "members": [
    {
      "skill": <name of the skill>,
      "description": <description of the skill>
    },
    ...
  ]
```

```
},
...
```

Figure 11: Cluster Skill Prompt

Cluster Attribute Prompt

You are an expert computer science curriculum designer with extensive experience in creating structured learning paths for algorithmic problem-solving.
Your goal is to take a large, potentially redundant list of attributes that affect the complexity of a coding problem and group overlapping or similar attributes into a set of canonical, non-overlapping attributes with clear, consolidated descriptions.

INPUT

You will be given a list of complexity attributes, where each attribute has an attribute name and a description of its definition and impact on problem complexity.
`{attribute_list}`

CLUSTERING REQUIREMENTS

Step 1: Group Similar Concepts

- Identify attributes that represent the same underlying complexity dimension.
- Look for semantic equivalence, not just naming similarities.
- Consider whether attributes would be varied together when creating problem mutations.

Step 2: Resolve Overlaps

- When attributes partially overlap, determine whether they should merge or remain distinct
- Preserve distinctions only if they represent independently variable complexity dimensions
- Eliminate redundancy while maintaining coverage

Step 3: Create Canonical Attributes

For each unified concept:
- Choose the most general and widely applicable name in **lower_snake_case**.
- Write a description that:
  - Defines what the attribute controls
  - Explains its impact on complexity when varied

Quality Criteria

- Each canonical attribute should be independently mutable
- No two attributes should be redundant or co-dependent
- Names should be intuitive and follow standard computer science terminology

OUTPUT FORMAT

Your final response must be a list of valid JSON objects exactly matching the schema below. Please output ONLY the list of JSON objects. Do not include any extra things.
List of JSON SCHEMA TO FOLLOW:

```
{
  <name_of_attribute>: <description_of_attribute>
},
...
```

Figure 12: Cluster Attribute Prompt

Skill Reflection Prompt

You are an expert Computer Science professor and a seasoned competitive programming coach.

TASK

Identify the necessary algorithmic skills used in the provided Python code snippet.
You are given a list of algorithmic skills and a Python code snippet. You need to identify all the algorithmic skills used in the code snippet.

REQUIREMENTS

- Review the provided skill list and the code snippet carefully and thoroughly.
- Use only the skills listed in the provided skill set. Avoid creating new skills or altering the existing skill names.

Each code snippet may involve multiple skills, but there is exactly one **core testing skill** that is the most important and necessary. If there is no other skills except the core testing skill, then the `other_skills` field should be an empty list.

OUTPUT FORMAT

Your final response **must be valid JSON** exactly matching the schema below.
Please output **ONLY** the JSON object. Do not include any extra things.

```
{
  "main_skill": <name_of_the_skill>,
  "other_skills": [
    <name_of_the_skill>,
    ...
  ]
}
```

INPUT

Skill List:
`{skill_list}`

Code Snippet:
`{code_snippet}`

Figure 13: Skill Reflection Prompt

Abduction Task Prompt

TASK

Create a new Python code snippet (custom classes allowed, which must be defined at the top of the snippet) with exactly one matching input.

FAILURE INFORMATION

`{failure_info}`

SKILLS

`{skill_str}`

CODE REQUIREMENTS

- The provided skill(s) should be the main testing concept of the code snippet.
- Name the entry function `f` (e.g., `def f(...): ...`). You may define nested definitions inside `f`.
- Ensure the function returns a value.
- Include at least one input parameter.
- Make the function deterministic.
- Make the snippet require state tracking across multiple data transformations, ensuring the task requires long multi-step reasoning.
- Avoid unnecessary function nesting; use simple, readable structure when possible. Only use nested functions if natural for the algorithm.
- **Avoid the following:**
  - Random functions or variables
  - Date/time operations
  - I/O operations (reading files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within 10 seconds on a modern CPU.
- All imports and class definitions should be at the very top of the code snippet.
- The snippet should end with a return statement from the main function `f`; anything after will be removed.

`{remove_input_from_snippet_prompt}{remove_after_return_prompt}`

INPUT REQUIREMENTS

- Provide exactly one test input for your function.
- Format multiple arguments with commas between them.
- Remember to add quotes around string arguments.

FORMATTING

Format your code with:

```
```python
def f(...):
    # your code here
    return ...
```
```

Format your input with:

```
```input
arg1, arg2, ...
```
```

EXAMPLE FORMAT

```
```python
def f(name: str, info: dict):
    # code logic here
```
```

````
    return result
```

```input
'John', {'age': 20, 'city': 'New York'}
```
````

Evaluation Criteria

- **Relevance (High Priority)**: The task should mainly examine the ability of the test subject to understand / reason / apply the skill(s).
- Executability: Your code should be executable given your input.
- Difficulty in reverse-engineering your `input` from (1) your `python` code and (2) the deterministic `output` obtained from your input.
- Creativity: The code must be sufficiently different from the provided reference snippets.
- Restricted usage of certain keywords and packages: You are not allowed to use the following words in any form, even in comments: `<|BANNED_KEYWORDS|>`.

Final Instruction

First, carefully devise a clear plan: how to make the skill(s) the main testing concept, how the snippet will be challenging and creative, and how to resolve any prior failure information. Then, write the final code snippet and its input.

Figure 14: Abduction Task Prompt

Abduction Task Mutation Prompt

Task

Create multiple variants of an original code reasoning task by systematically applying complexity attributes to increase difficulty and reasoning requirements.
The original task is a code reasoning deduction task that requires recovering a hidden input from a Python code snippet and its output.
You will be provided with:

- The original task (one code snippet and one output)
- The core algorithmic skill being tested
- A list of complexity attributes that may be applied

Your Mission

Step 1: Analyze the Original Task

Examine the provided task and identify:

- The current complexity level for each **applicable** attribute
- Opportunities to increase complexity while preserving solvability

Step 2: Generate Diverse Variants

Create **at least 10 distinct variants** using:

- Single-attribute mutations
- Multi-attribute mutations
- Comprehensive mutations applying all relevant attributes

Requirements for Each Variant

Code Requirements

- The provided skill(s) must remain the primary testing concept
- The entry function must be named `f`
- The function must return a value and accept at least one input parameter
- The code must be deterministic and free of randomness, I/O, logging, or external state
- Execution must complete within 10 seconds on a modern CPU
- All imports and class definitions must appear at the top of the snippet
- The snippet must end with a return statement from `f`

Input Requirements

- Provide exactly one hidden test input
- Format multiple arguments with commas
- Quote string arguments

Formatting

Code format:

````
```python
def f(...):
    # code here
    return ...
```
````

Input format:

````
```
arg1, arg2, ...
```
````
````

Complexity Enhancements

For each variant:

- Preserve the core algorithmic skill
- Increase difficulty meaningfully, not artificially
- Explain which complexity attributes were applied and why the variant is harder

OUTPUT FORMAT

Your final response must be valid JSON exactly matching the schema below. Output **ONLY** the JSON.

```
{
  "variant_1": {
    "complexity_attributes": [...],
    "description": "...",
    "code": "```python\n...\n```",
    "input": "```input\n...\n```"
  },
  ...
}
```

ORIGINAL TASK

Code:
`{code}`
Input:
`{input}`
Skill:
`{skill}`
Complexity Attributes:
`{complexity_attributes}`

Figure 15: Abduction Task Mutation Prompt

Abduction Task Crossover Prompt

TASK

Create a new Python code reasoning deduction task: a Python code snippet and one matching hidden input.
The task must use a novel combination of skills centered on the target core skill.
Custom classes are allowed and should be defined at the top of the snippet.

AVAILABLE INFORMATION

Skill Pool

A comprehensive list of algorithmic coding skills (with descriptions) you may choose from:
`{skill_pool}`

Target Core Skill

The primary skill that must be the central focus of your code snippet:
`{target_skill}`

Existing Skill Combinations

Previously created combinations of the target skill with other skills:
`{existing_combinations}`

YOUR TASK

You must create a **novel crossover** by combining the target skill with other compatible skills from the skill pool. This requires:

1. Examine existing combinations to understand what has already been done.
2. Identify skills from the pool that naturally work with the target skill but **have not been combined yet**.
3. Create a code snippet where all chosen skills are **essential and interconnected** to hide the true input while still producing the provided output.

The key challenge is to ensure your skill combination is both **NEW** (not in `existing_combinations`) and **MEANINGFUL** (skills genuinely complement each other, not forced).

CODE REQUIREMENTS

- The target skill **must** be the central concept; other skills should naturally support or enhance it.
- Name the entry function `f` (e.g., `def f(...): ...`); nested definitions inside `f` are allowed.
- Ensure the function returns a value.
- Include at least one input parameter.
- Make the function deterministic.
- Demonstrate **clear interdependence** between all skills in the combination (not isolated usage).
- Require state tracking across multiple data transformations to force multi-step reasoning.
- Avoid unnecessary function nesting; keep structure simple and readable unless nesting is natural for the algorithm.
- **Avoid:**

  - Random functions or variables
  - Date/time operations
  - I/O operations (reading files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within 10 seconds on a modern CPU.
- All imports and class definitions must be at the very top of the code snippet.
- The snippet must end with a `return` statement from the main function `f`. Anything after may be removed.
  `{remove_input_from_snippet_prompt}{remove_after_return_prompt}`

SKILL COMBINATION REQUIREMENTS

- The new combination must include the target skill plus **at least one** other skill from the skill pool.
- The combination must be meaningfully different from all existing combinations.
- Skills should work together **synergistically**, not just appear sequentially.
- You must justify why your chosen skills complement each other and how they work together to obfuscate the hidden input.

INPUT REQUIREMENTS

- Provide exactly one hidden test input for your function.
- Format multiple arguments with commas between them.
- Remember to add quotes around string arguments.

FORMATTING

Format your code as:

```
```python
def f(...):
    # your code here
    return ...
```
```

Format your input as:

```
```input
arg1, arg2, ...
```
```

OUTPUT FORMAT

Your final response **must be valid JSON** exactly matching the schema below.
Please output **ONLY** the JSON object. Do not include any extra text.

```
{
  "skill_combination": [<target_skill>, <new_combined_skill_1>,
  <new_combined_skill_2>, ...],
  "crossover_justification": <brief explanation of why and how these skills
  work well together>,
  "code": "```python\n<code snippet of the new skill combination>\n```",
  "input": "```input\n<input of the new skill combination>\n```"
}
```

EVALUATION CRITERIA

- **Novelty (Critical):** skill combination differs from all existing combinations.
- **Integration (High Priority):** all skills work together meaningfully.
- **Target Skill Focus:** target skill is the dominant concept being tested.
- **Executability:** code runs successfully with the provided input.
- **Difficulty:** reversing the hidden input requires understanding the interaction between skills.
- **Creativity:** scenario is distinct from existing snippets.
- **Complexity:** emphasize algorithmic reasoning/logic complexity (data structures, control flow, DP, recursion).
- **Restricted Keywords:** you cannot use the following words in any form: `<|BANNED_KEYWORDS|>`.

PROCESS

1. Analyze `existing_combinations` to identify gaps.
2. Propose a new skill combination with a clear justification.
3. Devise a plan for how the skills interact to conceal the input.
4. Implement the code snippet ensuring every skill is essential and requirements are met.
5. Provide the hidden input that exercises the full combination.

If there is previous failure information, address those issues explicitly in your new attempt.

Figure 16: Abduction Task Crossover Prompt

Abduction Task Predictor Prompt

TASK

Provide One Possible Input of a Python Code Snippet Given the Code and Output Given the following Code Snippet and the Output, think step by step then provide one possible input that produced the output. The input needs to be wrapped in “‘input“‘ tags. Remember if an argument is a string, wrap it in quotes. If the function requires multiple arguments, separate them with commas.

CODE SNIPPET

```
```python
{snippet}
```
```

OUTPUT

```
```output
{output}
```
```

OUTPUT FORMAT

```
```input
arg1, arg2, ...
```
```

Example Format

```
```input
'John', {{'age': 20, 'city': 'New York'}}
```
```

Figure 17: Abduction Task Predictor Prompt

Deduction Task Prompt

TASK

Create a New Python Code Snippet (where custom classes are allowed, which should be defined at the top of the code snippet) with one Matching

`{failure_info}`

Skills:
`{skill_str}`

CODE REQUIREMENTS

- The provided skill(s) should be the main testing concept of the code snippet.
- Name the entry function `f` (e.g., `def f(...): ...`), you can have nested definitions inside `f`
- Ensure the function returns a value
- Include at least one input parameter
- Make the function deterministic
- Make the snippet require state tracking across multiple data transformations, ensuring the task requires long multi-step reasoning.
- Avoid unnecessary function nesting; use simple, readable code structure when possible. Only use nested functions if it is natural for the algorithm
- AVOID THE FOLLOWING::
  - Random functions or variables
  - Date/time operations
  - I/O operations (files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within **10 seconds** on a modern CPU
- All imports and class definitions must be at the very top of the code snippet
- The snippet should end with a return statement from the main function `f`, anything after will be removed. `{remove_input_from_snippet_prompt}{remove_after_return_prompt}`

INPUT REQUIREMENTS

- Provide exactly one test input for your function
- Format multiple arguments with commas between them
- Remember to add quotes around string arguments

FORMATTING

- Format your code with:

```
```python
def f(...):
    # your code here
    return ...
```
```

- Format your input with:

```input
arg1, arg2, ...
```

Example Format:

```python
def f(name: str, info: dict):
    # code logic here
    return result
```

```input
'John', {{'age': 20, 'city': 'New York'}}
```

EVALUATION CRITERIA

- Relevance (High Priority), the task should mainly focus on examining the ability of the test subject to understand / reason / apply the skill(s)
- Executability, your code should be executable given your input
- Difficulty in predicting your `input` from 1) your `python` code and 2) the deterministic `output` that will be obtained from your `input`. Focus on either algorithmic reasoning or logic complexity. For example, you can define complex data structure classes and operate on them like trees, heaps, stacks, queues, graphs, etc, or use complex control flow, dynamic programming, recursions, divide and conquer, greedy, backtracking, etc
- Creativity, the code needs to be sufficiently different from the provided reference snippets
- Restricted usage of certain keywords and packages, you are not allowed to use the following words in any form, even in comments: `<|BANNED_KEYWORDS|>`

First, carefully devise a clear plan: e.g. how to make the skill(s) the main testing concept of the task, identify how your snippet will be challenging and creative. If there is previous failure information, think about how to resolve the failure and create acceptable code snippet. Then, write the final code snippet and its inputs.

Figure 18: Deduction Task Prompt

Deduction Task Mutation Prompt

TASK

Create Multiple Variants of an Original Task by Systematically Applying Complexity Attributes to Increase Complexity and Reasoning Requirements.

The original task is a code-reasoning deduction task that demands deep algorithmic reasoning to deduce the output from the input and Python code snippet.

You will be provided with:

- Original task: one Python code snippet and one input
- Skill: the algorithmic skill(s) that the original task mainly tests
- Complexity attributes: a list of complexity attributes you can consider to increase the complexity and reasoning requirements of the original task.

Be aware that not all attributes are applicable to the original task. You must first determine which attributes could be used.

YOUR MISSION

Step 1: Analyze the Original Task

Examine the provided task and identify:

- Current complexity level for each **applicable** attribute
- Potential for complexity enhancement in each dimension

Step 2: Generate Diverse Variants

Create **at least 10 different variants** via the following approaches:

- Single-attribute mutations: Enhance only one complexity dimension
- Multi-attribute mutations: Modify multiple but not all attributes at the same time
- Comprehensive mutations: Adjust all applicable attributes together

REQUIREMENTS FOR EACH VARIANT

Code Requirements

The provided skill(s) should be the main testing concept of the code snippet.

- Name the entry function `f` (e.g., `def f(...): ...`), you can have nested definitions inside `f`
- Ensure the function returns a value.
- Include at least one input parameter.
- Make the function deterministic.
- Make the snippet require state tracking across multiple data transformations, ensuring the task requires long multi step reasoning.

- Avoid unnecessary function nesting; use simple, readable code structure when possible. Only use nested functions if it is natural for the algorithm
- AVOID THE FOLLOWING:
  - Random functions or variables
  - Date/time operations
  - I/O operations (reading files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within 10 seconds on a modern CPU
- All imports and class definitions should be at the very top of the code snippet
- The snippet should end with a return statement from the main function 'f', anything after will be removed
- `{remove_input_from_snippet_prompt}{remove_after_return_prompt}`

Input Requirements

- Provide exactly one test input for your function
- Format multiple arguments with commas between them
- Remember to add quotes around string arguments.

Formatting

- Format your code with:

```
```python
def f(...):
    # your code here
    return ...
```
```

- Format your input with:

```
```input
arg1, arg2, ...
```
```

Example Format

```
```python
def f(name: str, info: dict):
    # code logic here
    return result
```

```input
'John', {{{{'age': 20, 'city': 'New York'}}}}
```
```

Complexity Enhancements

- Preserve the core algorithmic skill being tested.
- Consider the original task's difficulty (pass rate) when increasing complexity. The variants should not be too easy (pass rate = 100%) or too hard (pass rate = 0%).
- Each variant must be meaningfully different from others.
- Maintain solvability while increasing complexity.
- Output the complexity attributes that you considered for each variant and one concise description of how you increase the complexity by each attribute and why it is more complex than the original task.

Output Format

Your final response **must be valid JSON** exactly matching the schema below.
Please output **ONLY** the JSON object. Do not include any extra things. JSON SCHEMA TO FOLLOW:

```
{
  "variant_1": {
    "complexity_attributes": [
      <name of the complexity attribute being considered>,
      ...
    ],
    "description": <description of how variant_1 is more complex than the original task
    by the complexity attributes you considered>,
    "code": "```python\n<code snippet of variant_1>\n```",
    "input": "```input\n<input of variant_1>\n```"
  },
  "variant_2": ...,
  ...
}
```

Generation Strategy

1. **Breadth First:** Generate variants that explore different complexity attributes before combining them
2. **Difficulty Progression:** Include variants ranging from moderate increases to significant complexity jumps
3. **Reasoning Diversity:** Create variants that fail for different reasons (timeout, overflow, logic errors, edge cases).

Evaluation Criteria

- Relevance to core algorithmic skills: Each variant should test the same core skill.
- Complexity increases should be meaningful, not artificial.
- Variants should explore different failure modes and challenge different aspects of problem-solving.
- Executability: Each variant must be executable given its input.

Remember: The goal is to create a rich set of practice problems that gradually build expertise by challenging different aspects of the skill through varied complexity enhancements.

ORIGINAL TASK

Code:
`{code}`

Input:
`{input}`

Skill:
`{skill}`

Complexity Attributes:
`{complexity_attributes}`

Figure 19: Deduction Task Mutation Prompt

Deduction Task Crossover Prompt

TASK

Create a New Python Code Snippet (where custom classes are allowed, which should be defined at the top of the code snippet) with one Matching Input and with Novel Skill Combination

AVAILABLE INFORMATION

Skill Pool:
`{skill_pool}`
A comprehensive list of algorithmic coding skills along with their descriptions that you can choose from to create new combinations

Target Core Skill:
`{target_skill}`
The primary skill that must be the central focus of your code snippet along with its description

Existing Skill Combinations:
`{existing_combinations}`
Previously created combinations of the target skill with other skills

YOUR TASK

Create a novel crossover by combining the target skill with other compatible skills from the pool. This requires:

1. Examining existing combinations to understand what has already been done
2. Identifying skills from the pool that would naturally work with the target skill but haven't been combined yet
3. Creating a code snippet where all chosen skills are essential and interconnected to solve the problem

The key challenge is to ensure your skill combination is both NEW (not in existing combinations) and MEANINGFUL (skills genuinely complement each other rather than being artificially forced together).

CODE REQUIREMENTS

- The target skill MUST be the central concept, with other skills naturally supporting or enhancing it
- Name the entry function `f` (e.g., `def f(...): ...`), you can have nested definitions inside
- Ensure the function returns a value
- Include at least one input parameter
- Make the function deterministic
- The snippet should demonstrate clear interdependence between all skills in the combination (not just using skills in isolation)
- Require state tracking across multiple data transformations, ensuring multi-step reasoning
- Avoid unnecessary function nesting; use simple, readable code structure when possible
- AVOID THE FOLLOWING
  - Random functions or variables
  - Date/time operations
  - I/O operations (reading files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within 10 seconds on a modern CPU
- All imports and class definitions should be at the very top of the code snippet
- The snippet should end with a return statement from the main function `f`.
- `{remove_input_from_snippet_prompt}{remove_after_return_prompt}`

SKILL COMBINATION REQUIREMENTS

- Your combination must include the target skill plus at least one other skill from the skill pool
- The combination must be meaningfully different from all existing combinations

- Skills should work together synergistically, not just appear sequentially
- Justify why your chosen skills complement each other naturally and how they work together in your code snippet

INPUT REQUIREMENTS

- Provide exactly one test input for your function
- Format multiple arguments with commas between them
- Remember to add quotes around string arguments

FORMATTING

- Format your code with:

```
```python
def f(...):
    # your code here
    return ...
```
```

- Format your input with:

```
```input
arg1, arg2, ...
```
```

Example Format:

```
```python
def f(name: str, info: dict):
    # code logic here
    return result
```

```input
'John', {{'age': 20, 'city': 'New York'}}
```
```

OUTPUT FORMAT

Your final response **must be valid JSON** exactly matching the schema below. Please output **ONLY** the JSON object. Do not include any extra things. JSON SCHEMA TO FOLLOW:

```
{
  "skill_combination": [<target_skill>, <new_combined_skill_1>, <new_combined_skill_2>,

  ...],
  "crossover_description": <brief explanation of why and how these skills work well
  together>,
  "code": "```python\n<code snippet of the new skill combination>\n```",
  "input": "```input\n<input of the new skill combination>\n```"
}
```

EVALUATION CRITERIA

- Novelty (Critical): The skill combination must be different from all existing combinations
- Integration (High Priority): All skills must work together meaningfully, not just appear independently
- Target Skill Focus: The target skill should be the dominant concept being tested
- Executability: Your code should run successfully with the provided input
- Difficulty: The task should require understanding the interaction between skills, not just individual skill mastery
- Creativity: The scenario should be distinct from existing code snippets
- Complexity: Focus on algorithmic reasoning or logic complexity (e.g., complex data structures, control flow, dynamic programming, recursion)
- Restricted Keywords: You cannot use the following words in any form: `<|BANNED_KEYWORDS|>`.

PROCESS

1. First, analyze the existing combinations to understand patterns and identify gaps
2. Second, propose your new skill combination with clear justification
3. Third, devise a plan for how these skills will interact in your code
4. Fourth, implement the code snippet ensuring all skills are essential to the solution and following the code requirements
5. Finally, create an input that exercises the full combination

If there is previous failure information, address those issues explicitly in your new attempt.

Figure 20: Deduction Task Crossover Prompt

Deduction Task Predictor Prompt

TASK

Deduce the Output of a Python Code Snippet Given the Code and Input Given the following Code Snippet and the Input, think step by step then deduce the output that will be produced from plugging the Input into the Code Snippet. Put your output in "‘output“‘ tags. Remember if the output is a string, wrap it in quotes. If the function returns multiple values, remember to use a tuple to wrap them.

CODE SNIPPET

```
```python
{snippet}
```
```

INPUT

```
```input
{input_args}
```
```

Example Output:

```
```output
{'age': 20, 'city': 'New York'}
```
```

Figure 21: Deduction Task Predictor Prompt

Induction Task Prompt

TASK

Generate a natural coding problem related to the code snippet, and {num_inputs} inputs that can be plugged into the code snippet to produce a diverse set of outputs.

Using the code snippet provided below, design comprehensive {num_inputs} inputs that can be plugged into the code snippet to produce a diverse set of outputs. A subset of your given input and its deterministically produced outputs will be given to a test subject to deduce the function, which is meant to be an I.Q. test. You should also create a natural coding problem for which the given code snippet would be a valid solution, and your generated inputs would be the test inputs for the problem.

INPUT REQUIREMENTS

- Provide {num_inputs} valid inputs for the code snippet that comprehensively cover the code's behavior.
- For each input, format multiple arguments with commas between them
- Remember to add quotes around string arguments
- Each input should be individually wrapped in "‘input“‘ tags
- Ensure diversity: inputs should test different aspects and branches of the code

PROBLEM REQUIREMENTS

- Create a natural coding problem that clearly describes what needs to be solved. Do not include examples or constraints
- Write in an engaging, scenario-based style when possible (e.g., "You are given an array of meeting times..." or "A company needs to process customer orders...")
- The provided code snippet must be a correct and complete solution to the problem you describe
- Ensure that solving the problem statement would naturally lead to implementing logic similar to the given code snippet
- The problem statement should be wrapped in "‘problem“‘ tags
- You cannot include or leak the code snippet in the problem statement

FORMATTING

- Format your input with:

```
```input
arg1, arg2, ...
```
```

Example Format:

```
```input
'John', {{'age': 20, 'city': 'New York'}}
```
```input
'Sammy', {{'age': 37, 'city': 'Los Angeles'}}
```
```

EVALUATION CRITERIA

- Executability, the code should be executable given your inputs
- Coverage, the inputs should cover the whole input space of the code snippet
- Creativity, the inputs need to be sufficiently different from each other
- The overall selection of inputs and message combined should be challenging for the test subject, but not impossible for them to solve
- Problem Quality, The problem statement should read like a real coding assessment question

PROCESS

First, carefully devise a clear plan: e.g. understand the code snippet, then identify how your proposed inputs have high coverage, and why the inputs will be challenging and creative. Then, write the inputs and message. Remember to wrap your inputs in "‘input“‘ tags, and your message in "‘message“‘ tags.

CODE SNIPPET

```python
{code}
```

Figure 22: Induction Task Prompt

Induction Task Hint Prompt

TASK

Generate progressive hints for a coding problem.

The coding problem below was generated based on a code snippet and has proven too challenging for test subjects to solve. Your task is to create a series of progressive hints that guide the solver toward the solution WITHOUT giving away the complete answer

ORIGINAL PROBLEM

```problem
{problem}
```

CODE SNIPPET

```python
{code}
```

HINTS REQUIREMENTS

- Generate 3–4 progressive hints that gradually reveal the solution approach.
- Follow the hint style:
  - Start with high-level intuition or pattern recognition
  - Progress to algorithm/data-structure suggestions
  - Then provide implementation details or key insights
  - Final hint can outline the approach at a coarse level but should not reveal the actual code implementation
- Each hint should build upon the previous one
- Hints should be concise, to the point, and guide thinking without revealing the exact solution
- Ensure the hints are grounded to the code snippet, which is the solution to the problem
- Wrap each hint in “‘hint“‘ tags.

FORMATTING

- Format your hints with:

```hint
<hint_content>
```

Example Format:

“‘hint Can you think of this problem in terms of a decision tree, where at each step we have n decisions, where n is the size of the array? “‘
“‘hint We can use backtracking to recursively traverse these paths and make decisions to choose an element at each step. “‘

EVALUATION CRITERIA

- Progressive Difficulty: Each hint should reveal slightly more than the previous
- Guidance Quality: Hints should genuinely help stuck test subjects to make progress, not just give answers
- Clarity: Use simple language and clear explanations

First, carefully devise a clear plan: e.g. understand the code snippet, then identify what concepts or patterns might not be obvious, what edge cases could trip up solvers, what algorithmic insight is needed?. Then, create the hints. Remember to wrap your hints in “‘hint“‘ tags.

Figure 23: Induction Task Hint Prompt

Induction Task Predictor Prompt

TASK

Deduce the Function that Produced the Outputs from the Inputs

Given a set of input/output pairs and a problem that describes the function, think through the problem step by step to deduce a general code snippet. This code should produce the hidden outputs from the hidden inputs, matching the original data-generating code that created the input/output pairs. Place your final answer inside python tags! It may be helpful to work through each input/output pair individually to test your function. If your function doesn’t work as expected, revise it until it does. The final code snippet will be used to evaluate your response, which is wrapped in “‘python“‘ tags.

CODE REQUIREMENTS

- Name the entry function `f` (e.g., `def f(...): ...`), you can have nested definitions inside ‘`f`‘
- Ensure the function returns a value
- Include at least one input parameter
- Make the function deterministic
- AVOID THE FOLLOWING:
  - Random functions or variables
  - Date/time operations
  - I/O operations (reading files, network requests)
  - Printing or logging
  - Any external state
- Ensure execution completes within 10 seconds on a modern CPU
- All imports and class definitions should be at the very top of the code snippet
- The snippet should end with a return statement from the main function ‘`f()`‘, anything after will be removed

INPUT AND OUTPUT PAIRS

`{input_output_pairs}`

PROBLEM

`{problem}`

EXAMPLE OUTPUT

```python
def f(a):
    return a
```

Name your entry function `f()`.

Figure 24: Induction Task Predictor Prompt